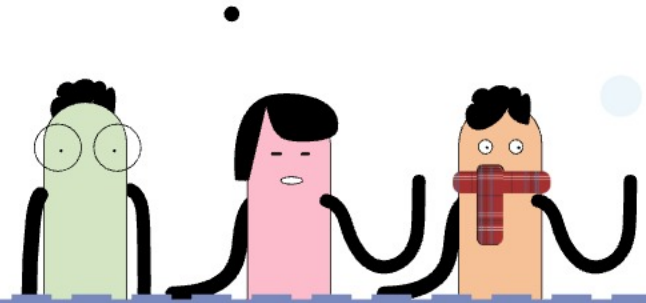


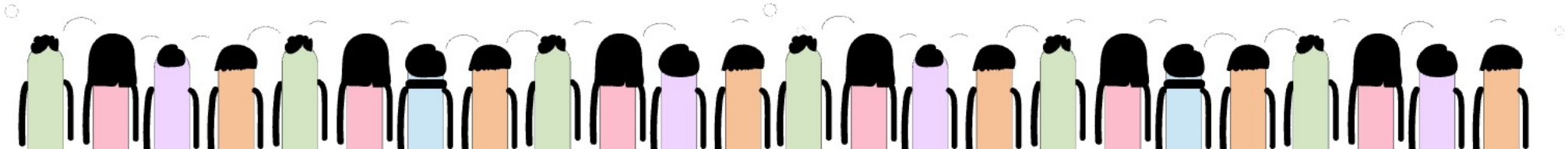
Firmware product engineer

Yocto/Bit Bake

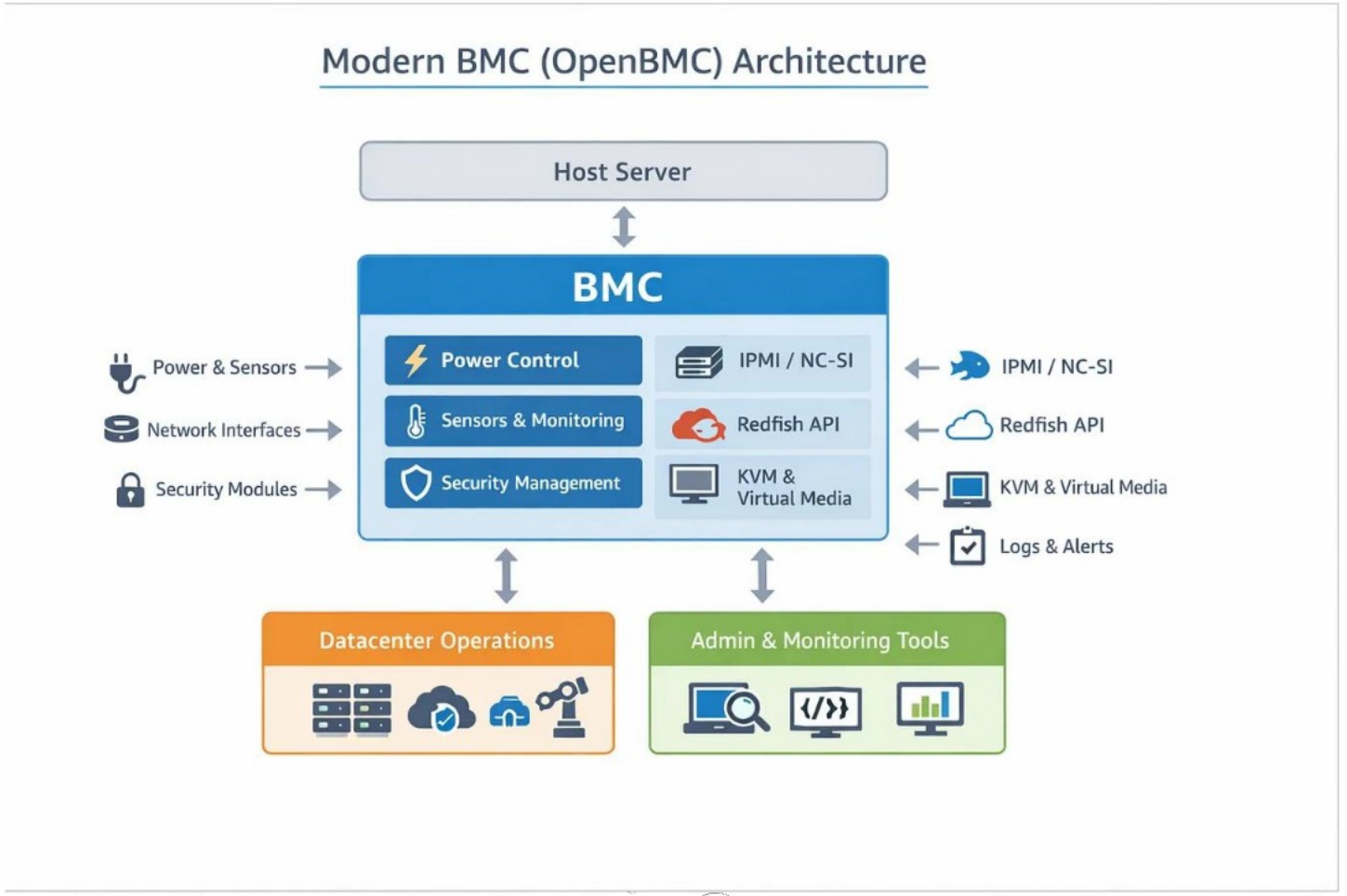
BMC

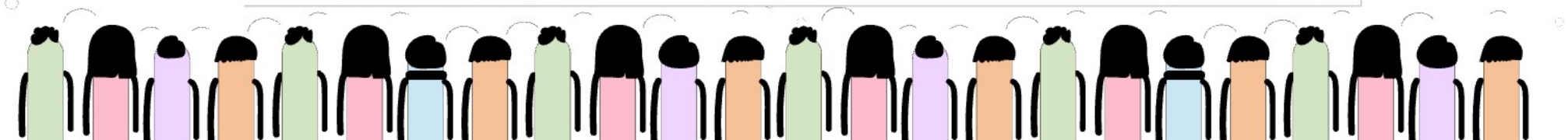
What is a Baseboard Management Controller (BMC)?

A **Baseboard Management Controller (BMC)** is a specialized microcontroller embedded on a server motherboard that provides **out-of-band remote management** capabilities for enterprise servers. It operates independently from the main CPU and operating system, allowing administrators to monitor, manage, troubleshoot, and recover systems remotely — even when the server OS is powered off, crashed, or unresponsive. supermicro.com +2



BMC


Modern BMC (OpenBMC) Architecture
Host Server
BMC
Power Control
Sensors & Monitoring
Security Management
IPMI / NC-SI
Redfish API
KVM & Virtual Media
Power & Sensors
Network Interfaces
Security Modules
IPMI / NC-SI
Redfish API
KVM & Virtual Media
Logs & Alerts
Datacenter Operations
Admin & Monitoring Tools



BMC

In real enterprise products, the BMC acts like a:

- dedicated embedded Linux computer
- always-on management processor
- hardware monitoring controller
- remote recovery engine

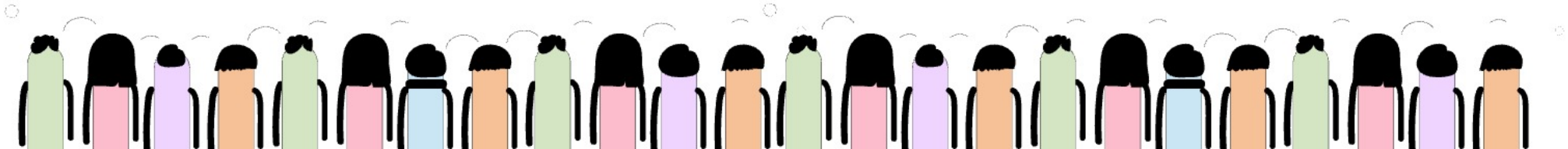
inside the server.

The BMC has:

- its own ARM processor
- memory
- network stack
- firmware
- Linux-based services

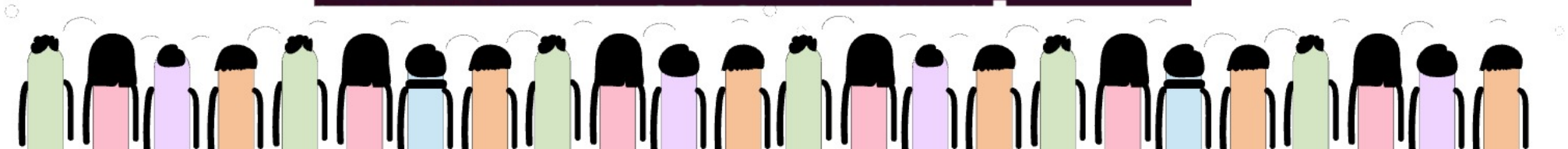
Real Hardware Flow

```
Enterprise Server
│
├── Main CPU (Intel/AMD)
├── Main OS (Linux/Windows/VMware)
│
└── BMC Subsystem
    ├── ARM Processor
    ├── Embedded Linux
    ├── Sensor Interfaces
    ├── Network Interface
    └── Remote Management Services
```



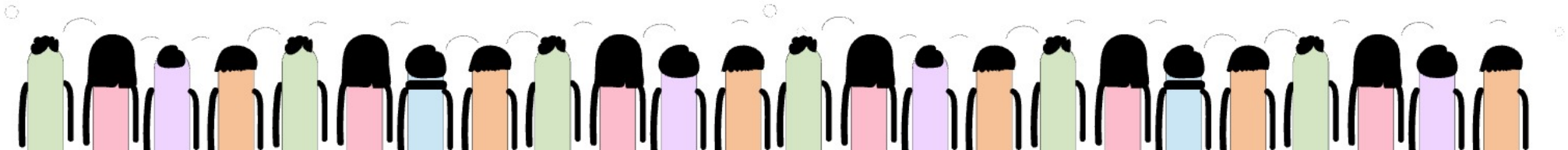
Linux Basics

```
421  sudo apt update
422  arm-linux-gnueabihf-gcc --version
423  sudo apt update
424  cd /etc/apt
425  ls
426  cat sources.list
427  nano sources.list
428  ls
429  cd ..
430  ls
431  cd ..
432  ls
433  cat smile.i
434  ls
435  cat smile.s
436  ls
437  gcc -c smile.s -o smile.o
438  ls
439  cat smile.o
440  file smile.o
441  gcc smile.o -o a.out
442  ls
443  ./a.out
444  nm smile.o
445  readelf -a a.out
446  ls
447  history
```



Large-Scale Server Management

Component	Technology
Bootloader	U-Boot
OS	Embedded Linux
Build System	Yocto
APIs	Redfish/IPMI
IPC	D-Bus
Logging	journald
Services	systemd
Firmware Stack	OpenBMC/MegaRAC



FIRMWARE

1. FIRMWARE

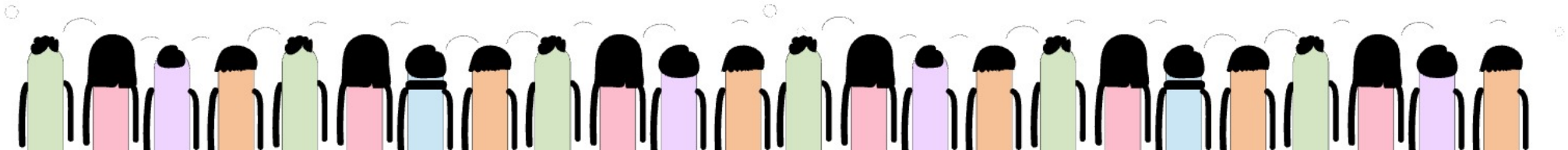
Real Production Definition

Firmware is low-level software stored in non-volatile memory (flash/ROM) that initializes hardware and provides basic control before the operating system starts.

Firmware directly interacts with hardware components such as:

- CPU
- memory
- storage
- sensors
- network controllers
- power systems

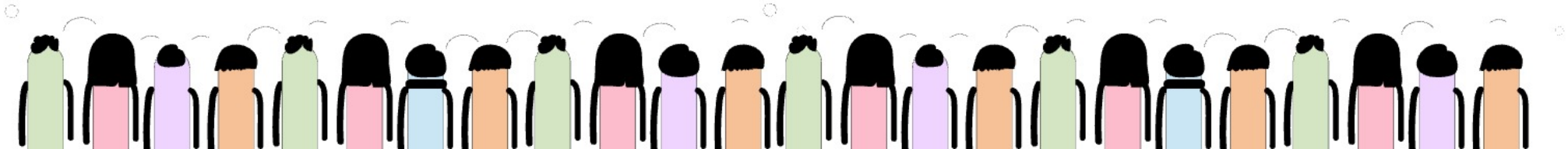
Firmware is tightly coupled to hardware. (redhat.com ↗)



FIRMWARE

Real Production Examples

Product Type	Real Firmware
Enterprise Server	HPE iLO Firmware
BIOS/UEFI	AMI Aptio UEFI
SSD Controller	NVMe Firmware
Router	Cisco IOS-XE firmware
BMC	OpenBMC
GPU	NVIDIA GPU firmware



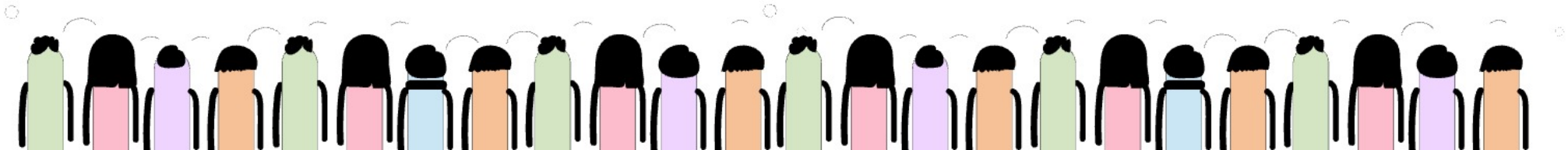
FIRMWARE

What Firmware Does

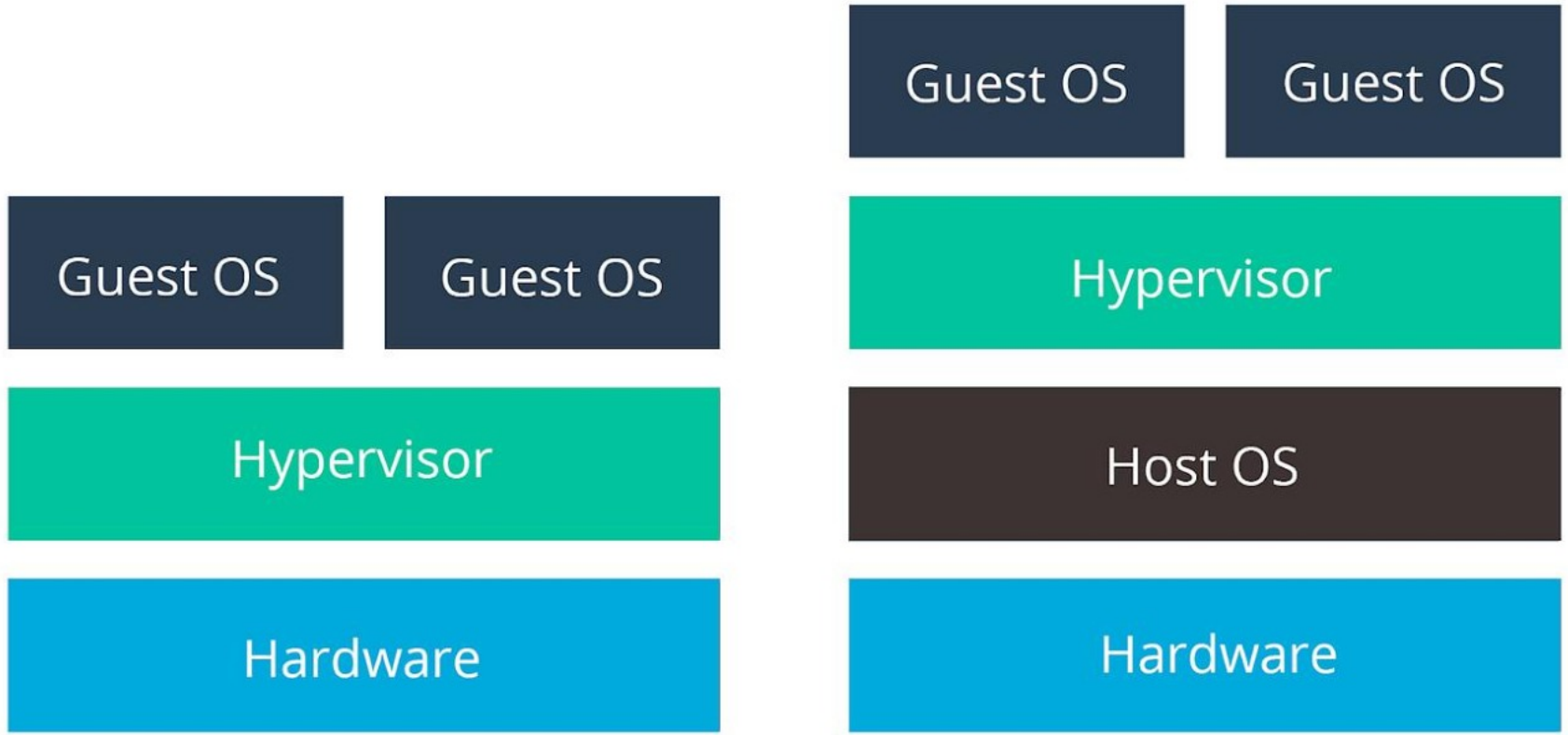
Firmware responsibilities:

- hardware initialization
- boot sequence control
- thermal management
- power management
- hardware diagnostics
- secure boot validation

```
Power ON
   ↓
Firmware Starts
   ↓
CPU + RAM Initialization
   ↓
Hardware Checks
   ↓
Load Bootloader/OS
```

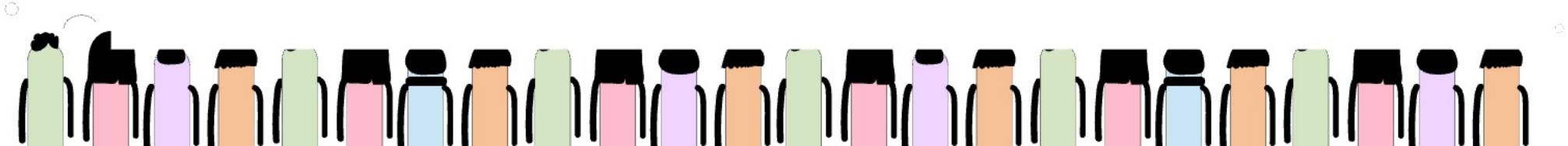


HYPERVISOR



TYPE 1 HYPERVISOR

TYPE 2 HYPERVISOR

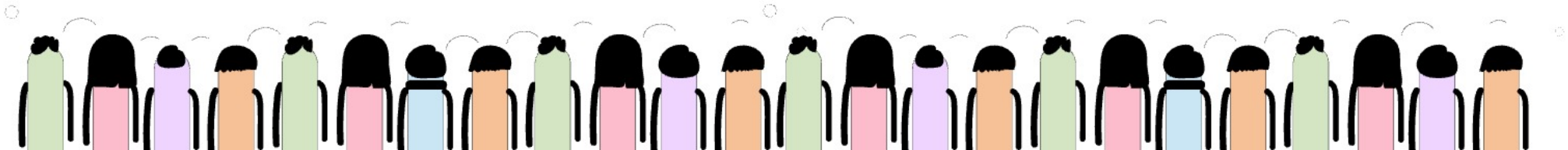


HYPERVISOR

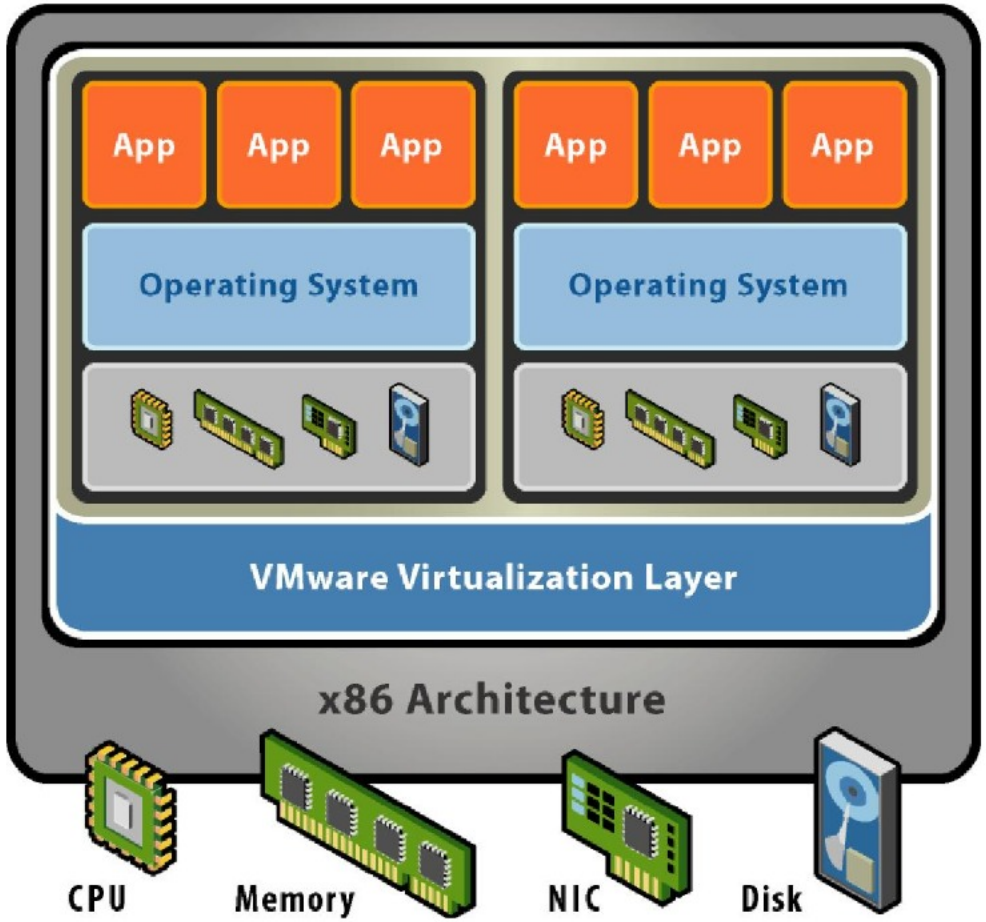
Real Production Definition

A Hypervisor is virtualization software that creates and manages multiple virtual machines (VMs) on a single physical server.

It abstracts physical hardware and allows multiple operating systems to run simultaneously on one machine. (vmware.com ↗)



VMWARE



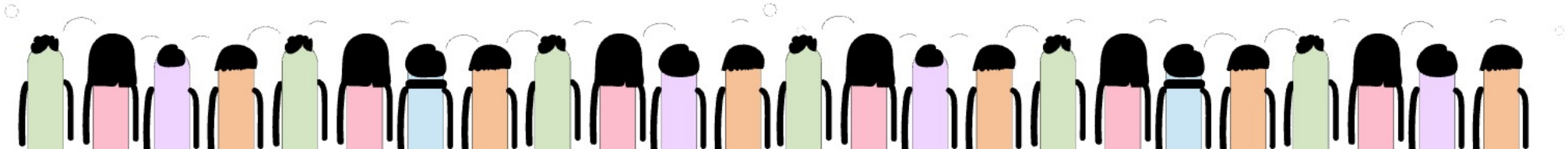
Real Production Example

In enterprise servers:

```
Firmware
   ↓
Linux/Windows OS
   ↓
Applications
```

Example:

- firmware initializes hardware
- Linux boots
- banking application runs

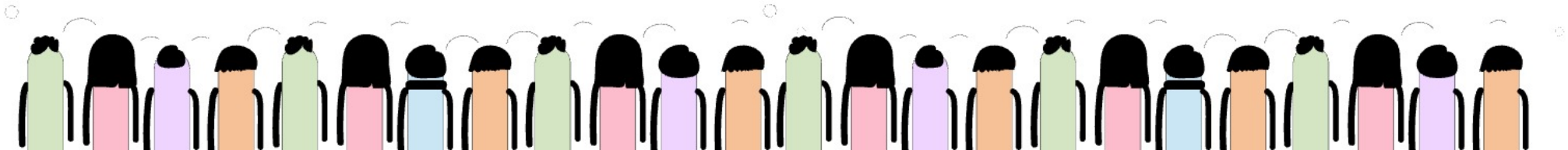


BOOT PROCESS

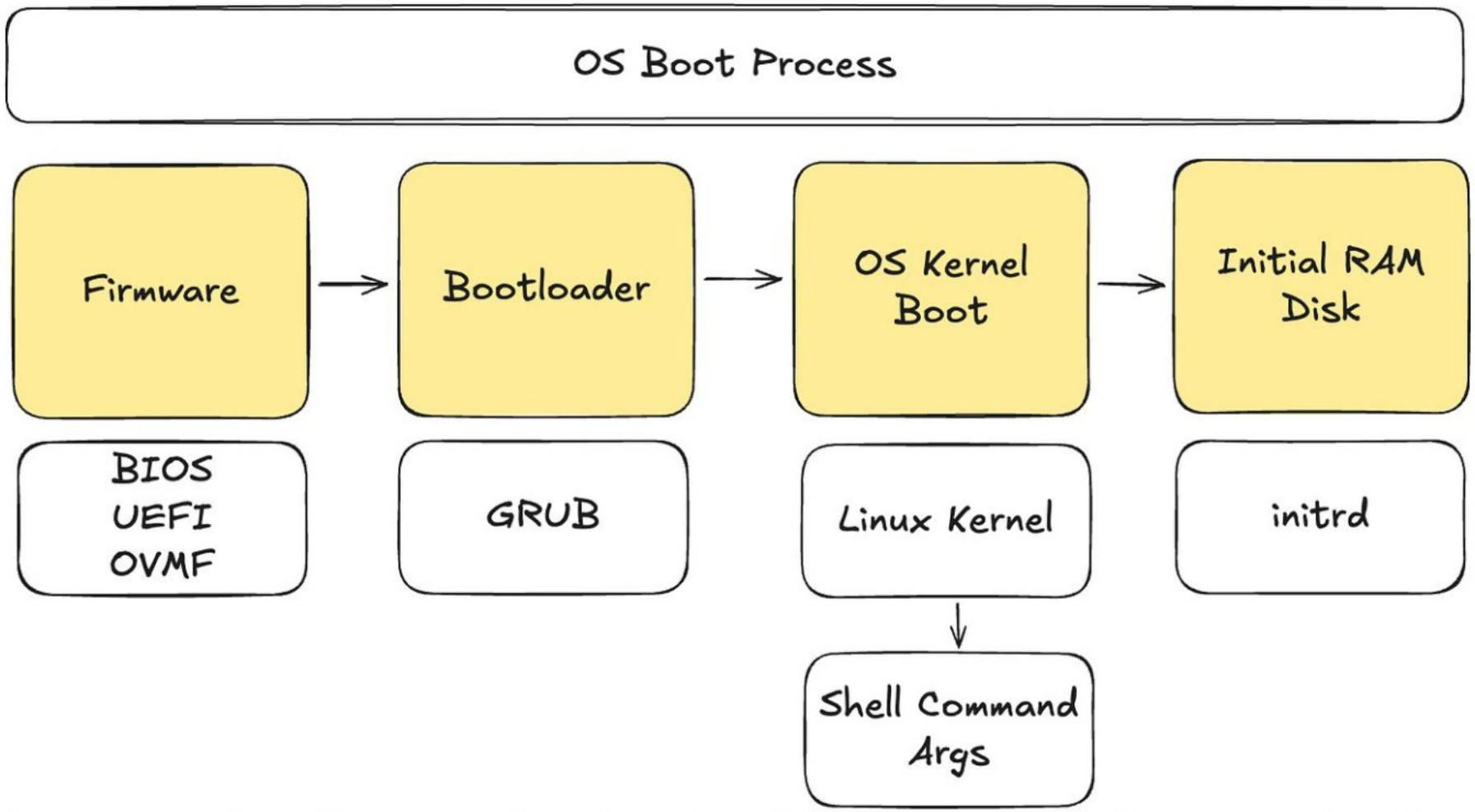
An Operating System (OS) is system software that manages hardware resources and provides services for applications and users.

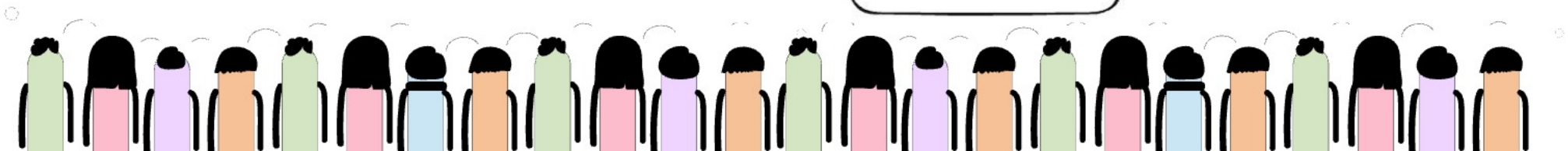
The OS sits above firmware and provides:

- process management
- memory management
- networking
- storage management
- file systems
- device abstraction

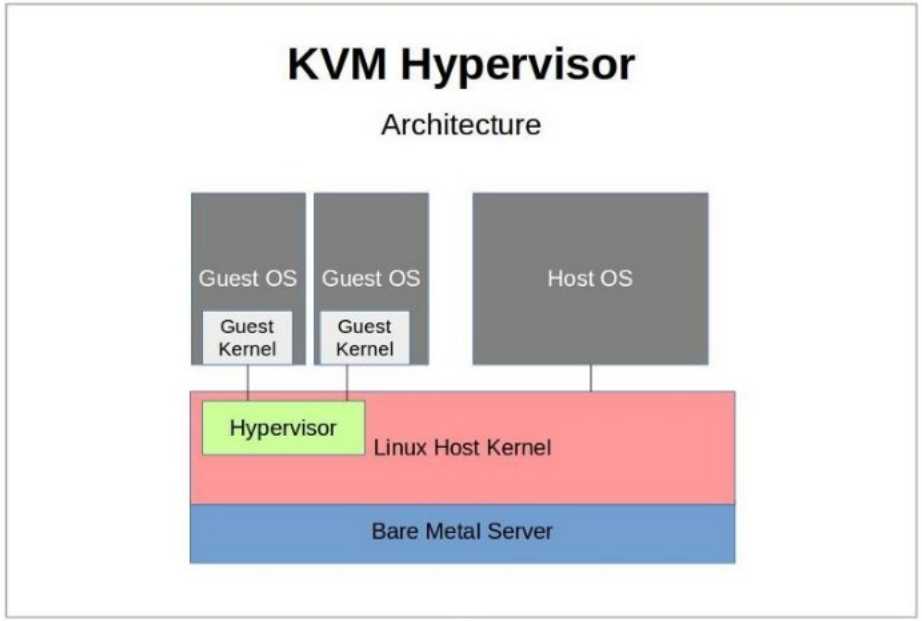


BOOT PROCESS





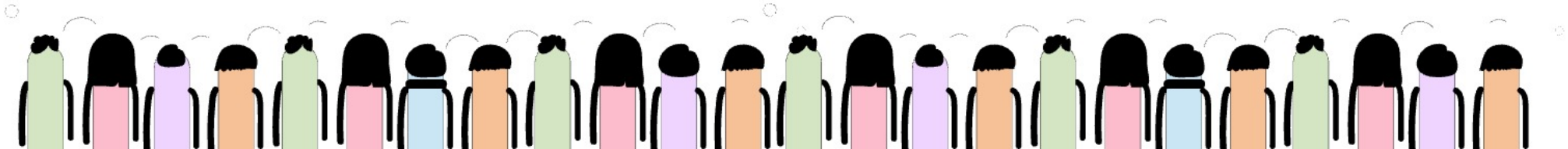
HYPERVISOR



What Hypervisor Does

Hypervisor responsibilities:

- create virtual machines
- allocate CPU/memory/storage
- isolate workloads
- virtualize hardware
- manage VM lifecycle



HYPERVISOR

Real Enterprise Example

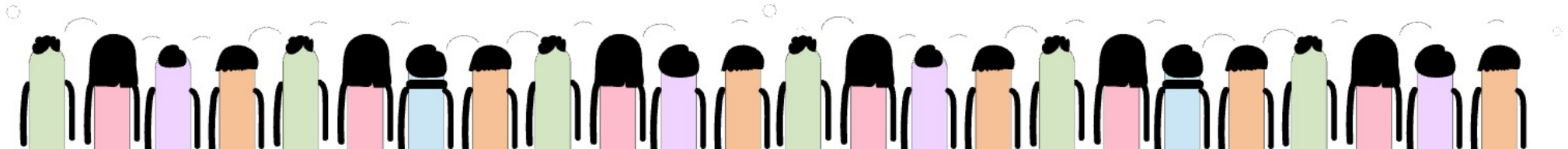
One physical server may run:

- banking VM
- database VM
- AI workload VM
- monitoring VM

using hypervisor virtualization.

Real Stack Example

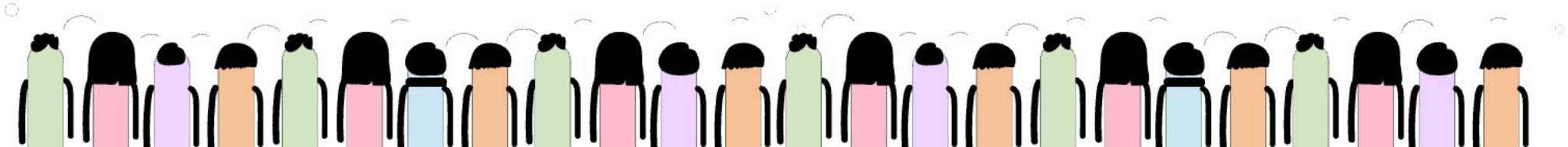
```
Firmware
   ↓
Hypervisor (VMware ESXi)
   ↓
Virtual Machines
   ├── Linux VM
   ├── Windows VM
   └── Database VM
```



End-to-End Boot Flow

End-to-End Boot Flow

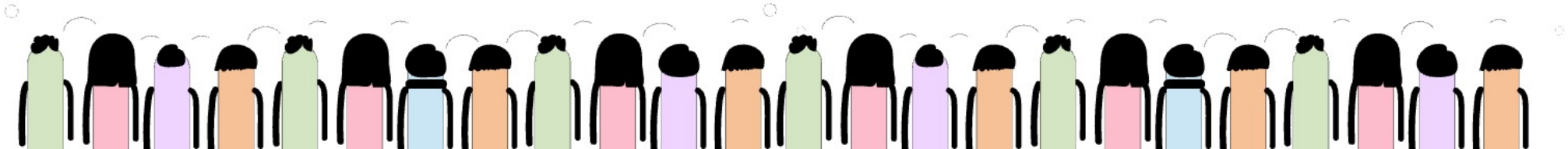
```
Power ON
   ↓
Firmware Starts
   ↓
Hardware Initialized
   ↓
Hypervisor Loads
   ↓
Virtual Machines Start
   ↓
Operating Systems Boot
   ↓
Applications Run
```



End-to-End Boot Flow

Real Enterprise Use Cases

Layer	Real Use Case
Firmware	Remote server recovery
OS	Running enterprise applications
Hypervisor	Running multiple VMs



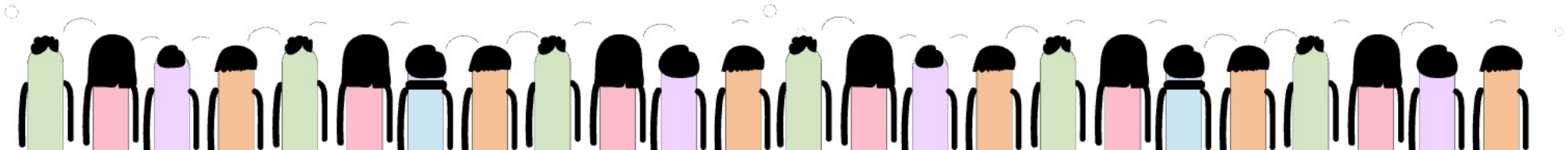
Native Compilation

In real production systems such as:

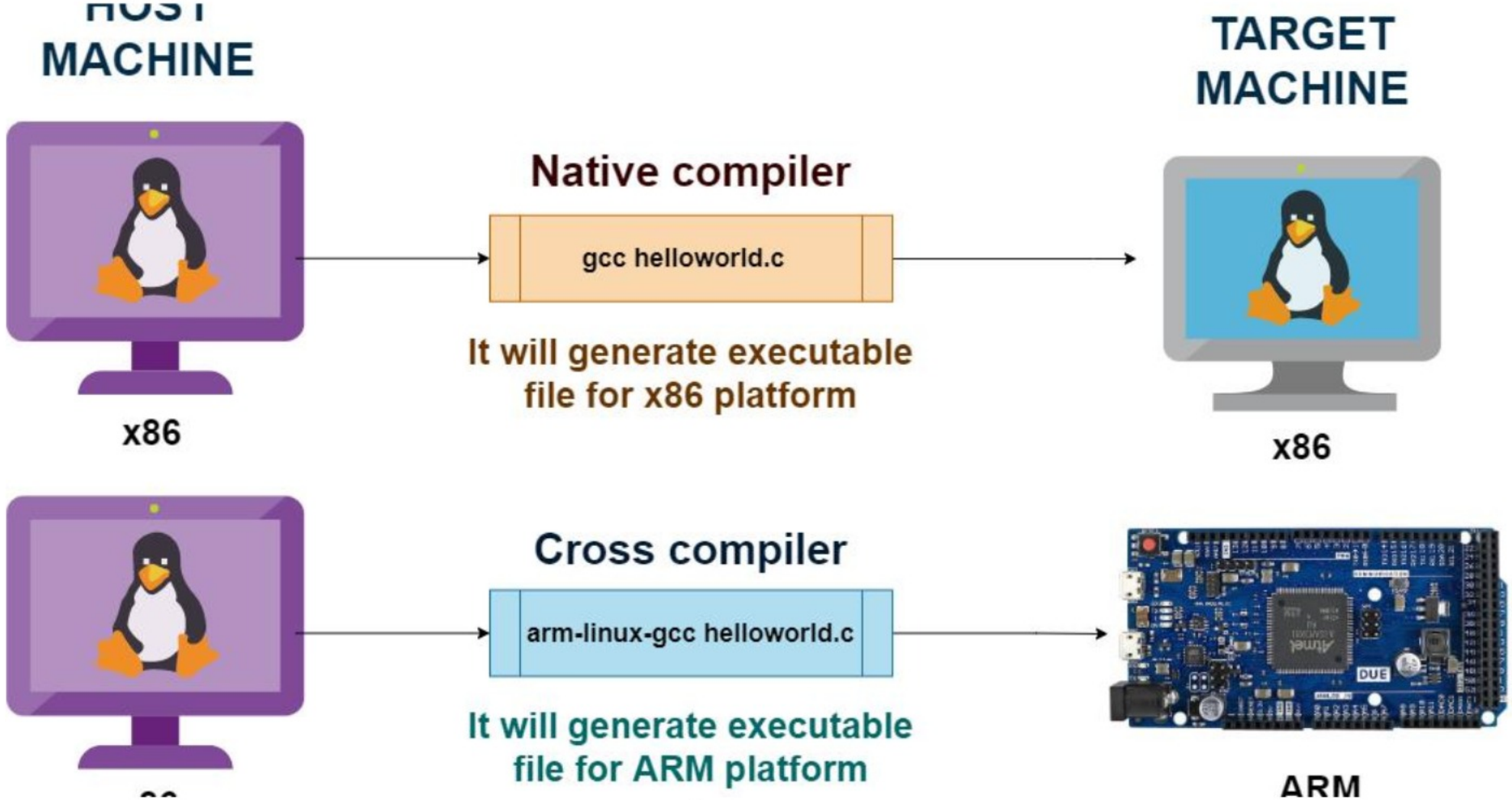
- enterprise firmware
- embedded Linux devices
- BMC platforms
- automotive ECUs
- telecom equipment
- routers
- IoT systems

developers often build software for hardware architectures different from their own development machine.

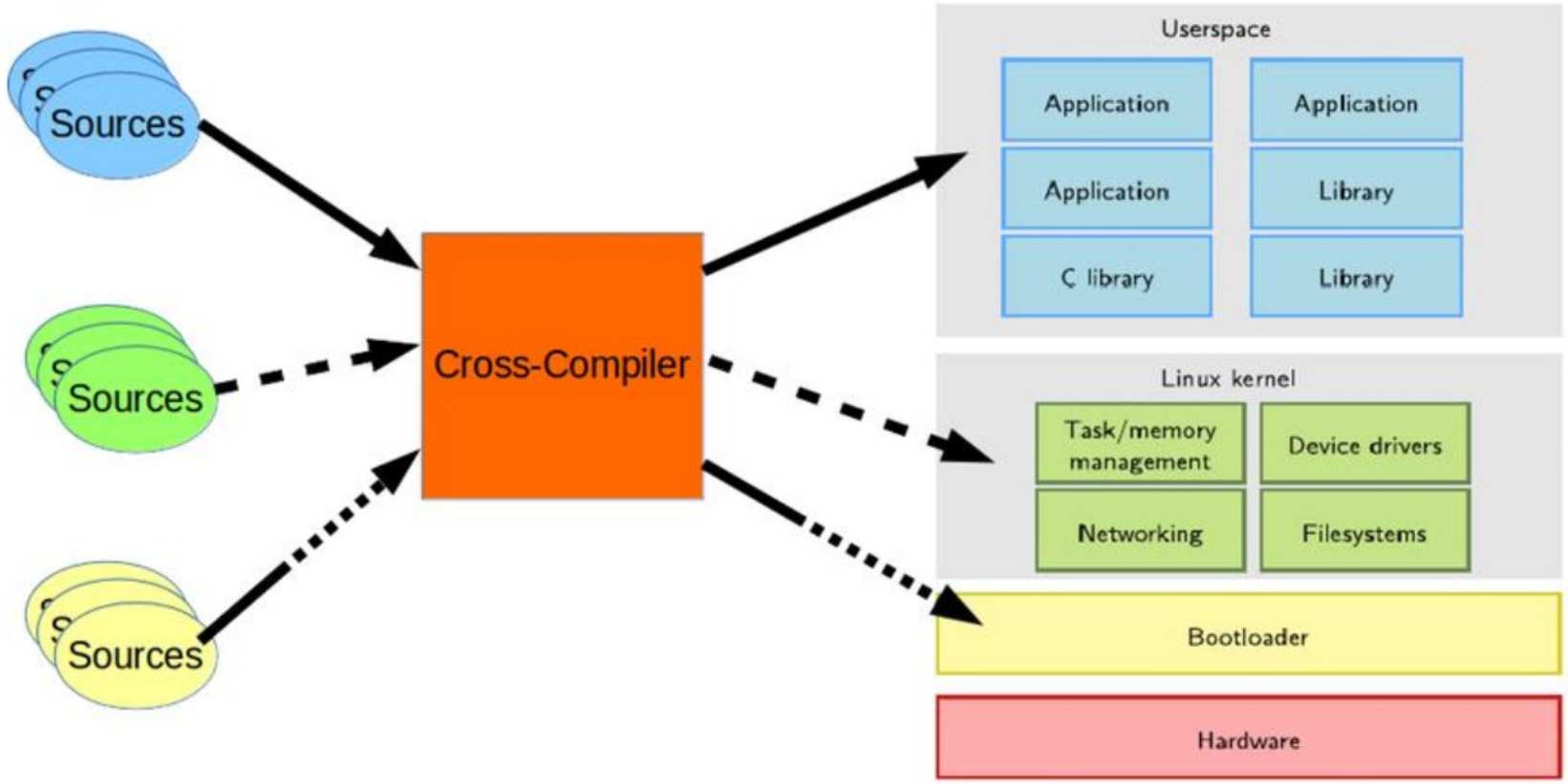
This is where native compilation and cross compilation become critical concepts.

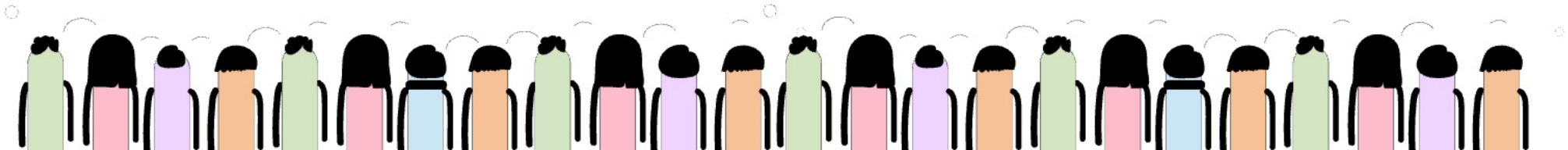


Native Compilation

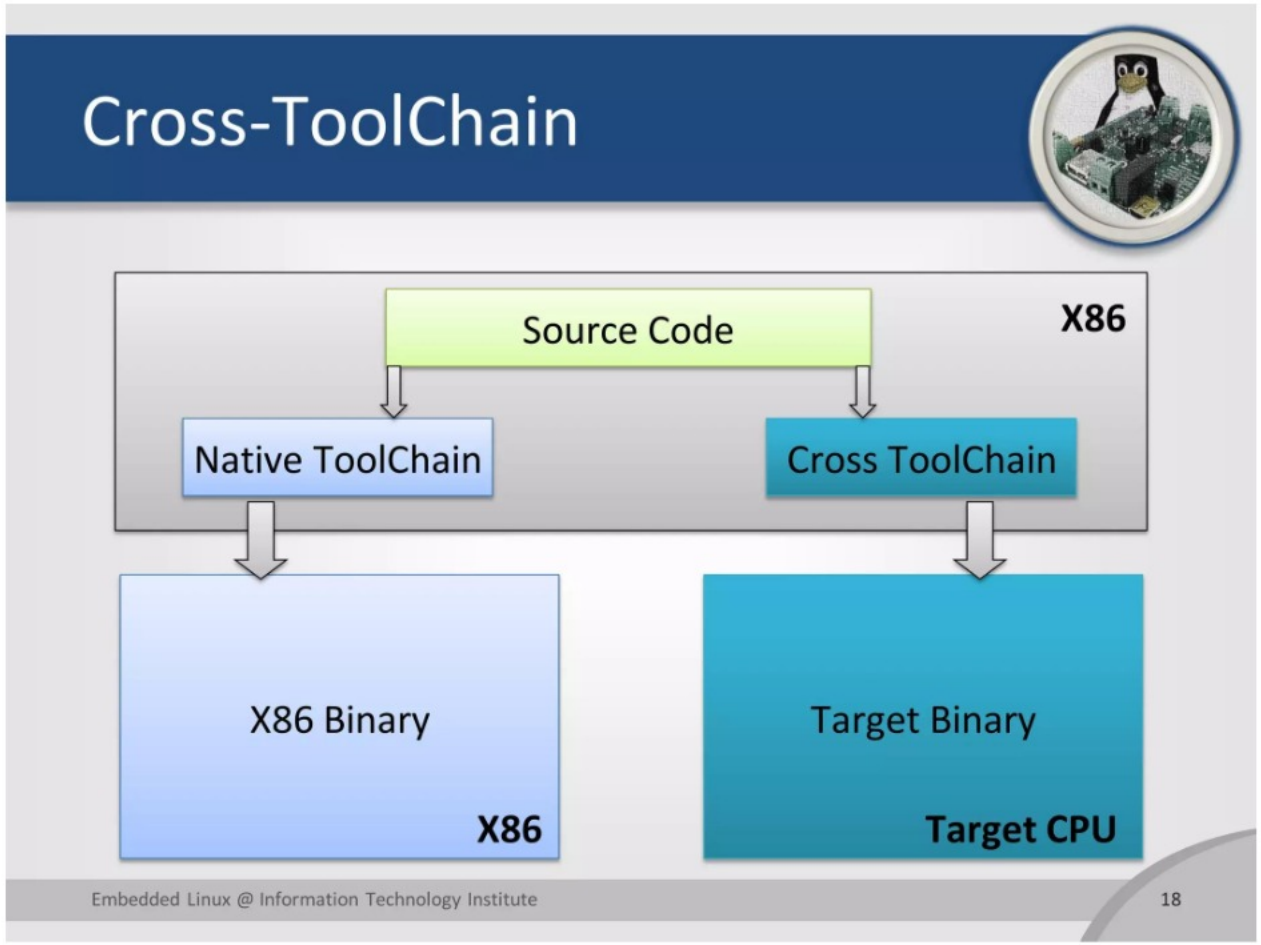


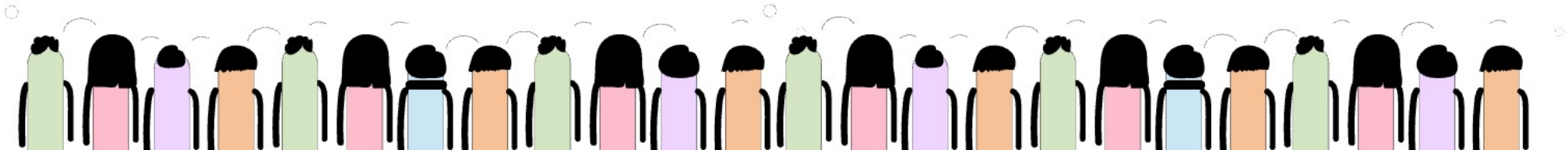
Cross Compiler





Cross Compiler



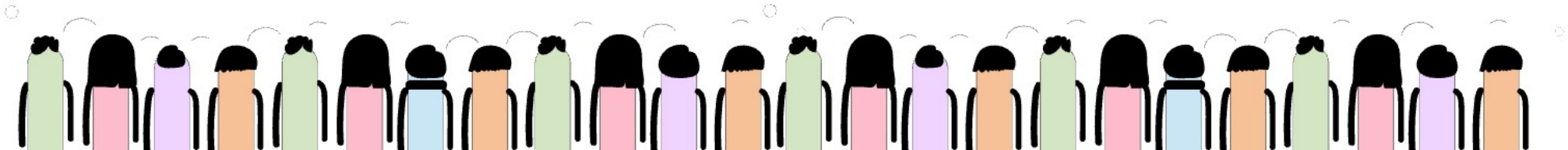


Native Compiler

Where Native Compilation Is Used

Native compilation is common in:

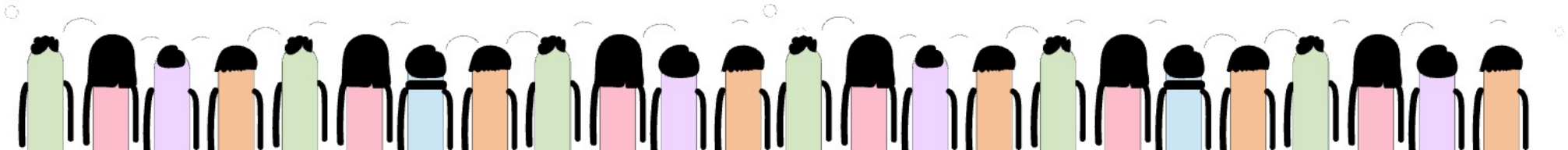
- desktop software
- backend applications
- enterprise Java services
- local Linux utilities
- development tools



Native Compiler

Real Embedded Linux Flow

```
x86 Build Server
   ↓
ARM Cross Compiler
   ↓
ARM Firmware Binary
   ↓
BMC/iLO Device
```

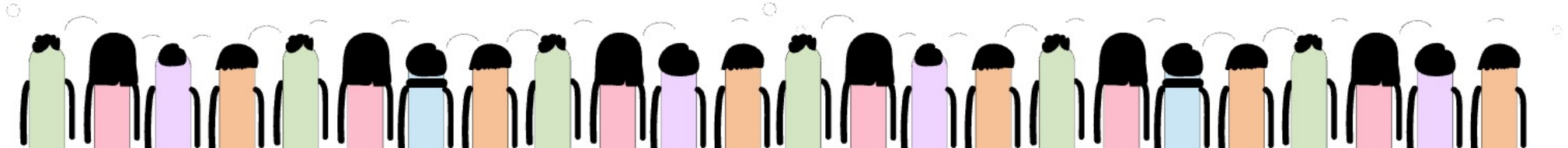


Native Compiler

Real Production Areas Using Cross Compilation

Cross compilation is heavily used in:

Industry	Examples
Enterprise Servers	BMC firmware
Automotive	ECU firmware
Telecom	Base station firmware
Networking	Router OS
Consumer Electronics	Smart TVs
Aerospace	Embedded avionics
IoT	Sensor gateways



Cross Compiler

Native Compilation Example

```bash
gcc hello.c -o hello
```

Generated binary:

```
x86 Linux executable
```

Runs directly on same machine.

Real Example

```
Development Machine:
x86 Ubuntu Linux

Target Machine:
x86 Ubuntu Linux
```

Both are same architecture.

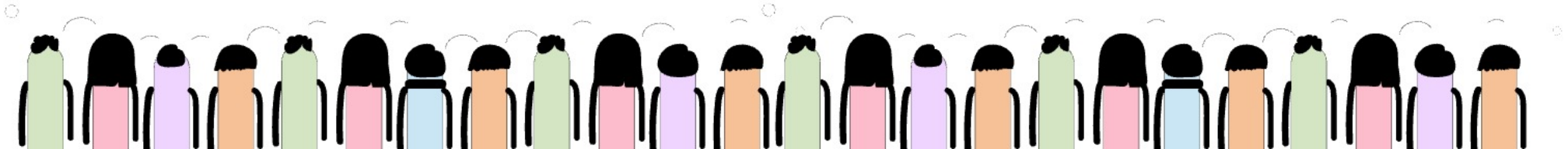
Compilation happens locally.

1. Native Compilation

Real Production Definition

Native compilation means:

> compiling software on the same architecture and operating system where the application will run.

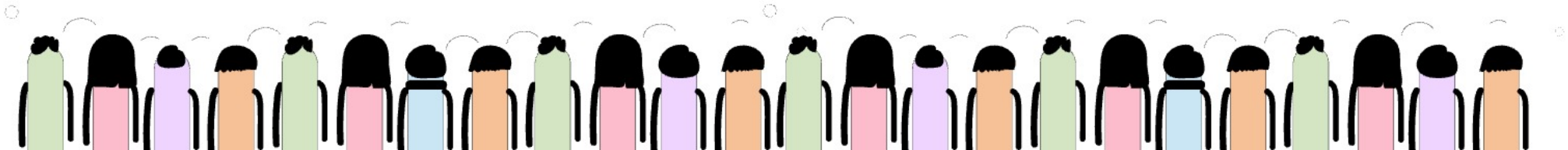


GCC

Real Production Definition

GCC is the compiler that converts source code into machine code for a target architecture.

(gcc.gnu.org ↗)



GCC

Real Responsibilities

GCC performs:

- preprocessing
- compilation
- optimization
- assembly generation

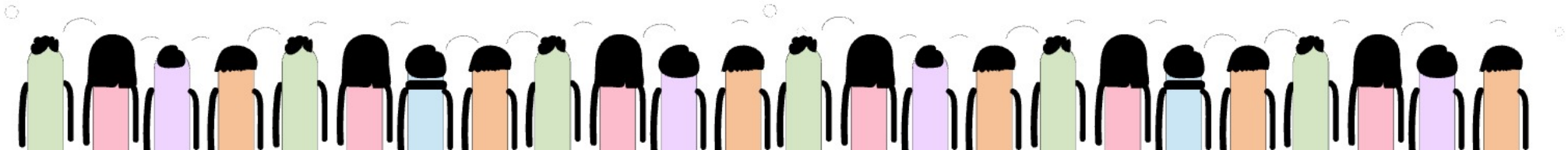
Real Example

Bash

```
arm-linux-gnueabihf-gcc hello.c -o hello_arm
```

This produces:

```
ARM executable binary
```



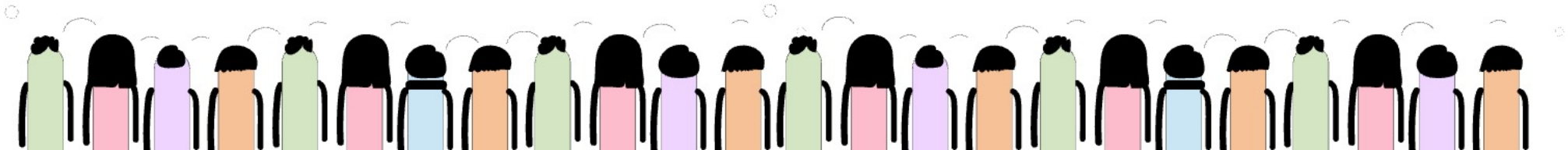
binutils

Real Production Definition

binutils is a collection of binary utilities used for:

- linking
- assembling
- analyzing binaries
- symbol inspection

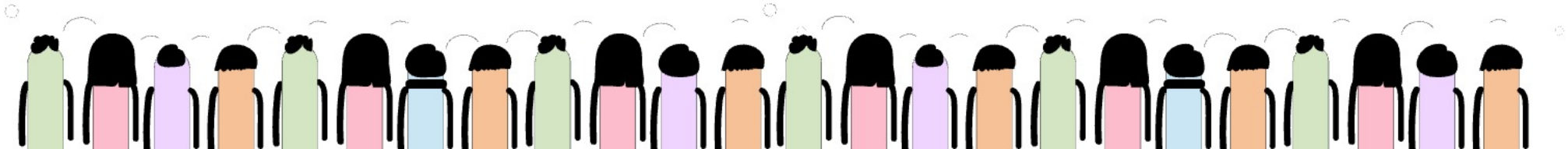
(sourceware.org ↗)



binutils

Important binutils Components

Tool	Purpose
ld	Linker
as	Assembler
objdump	Binary analysis
nm	Symbol inspection
strip	Remove debug symbols



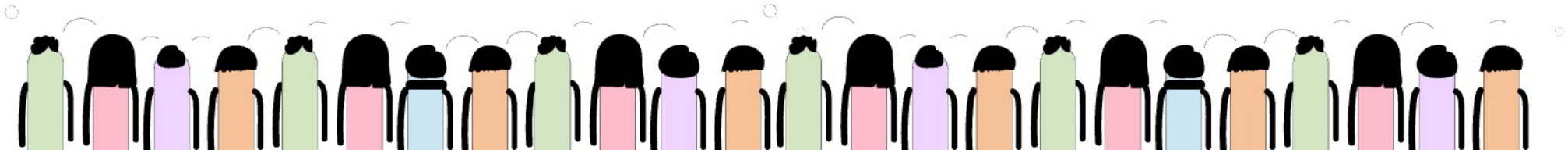
binutils

Real Example

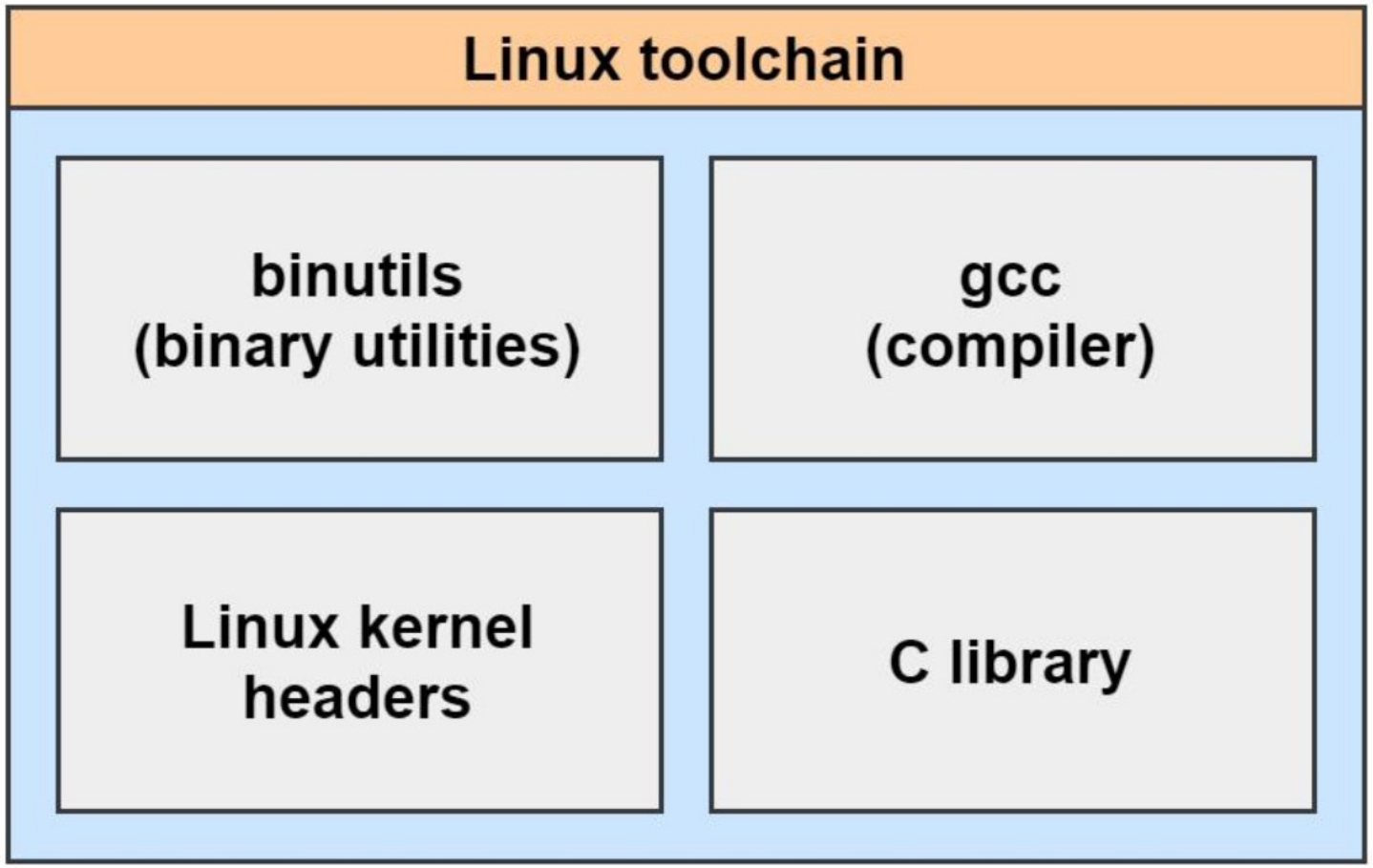
Bash

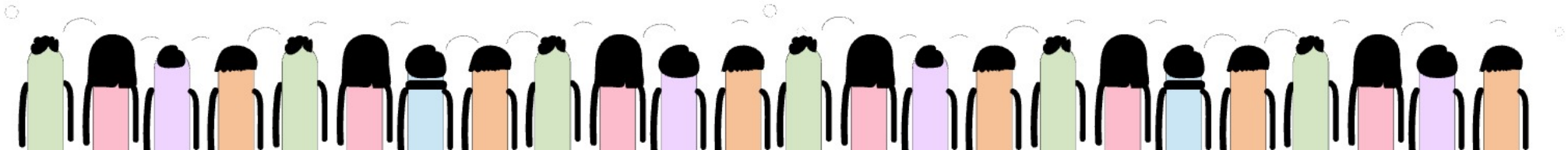
```
arm-linux-gnueabihf-objdump -d hello_arm
```

Disassembles ARM binary.



binutils


Linux toolchain
binutils
(binary utilities)
gcc
(compiler)
Linux kernel
headers
C library



libc

Real Production Definition

libc provides standard runtime functions required by C applications.

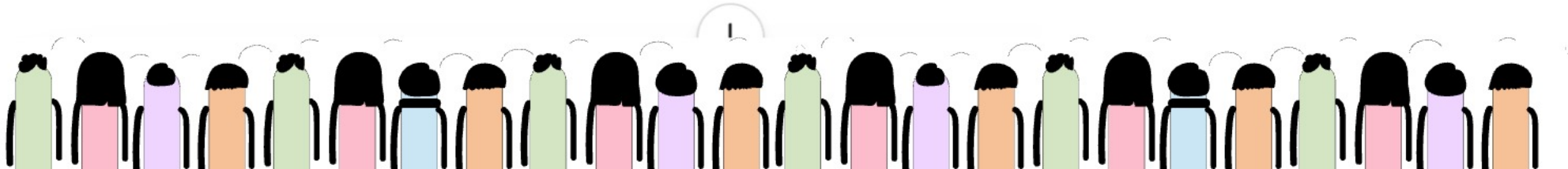
(gnu.org ↗)

Examples of libc Functions

‹/› C

```c
printf()
malloc()
fopen()
memcpy()
```

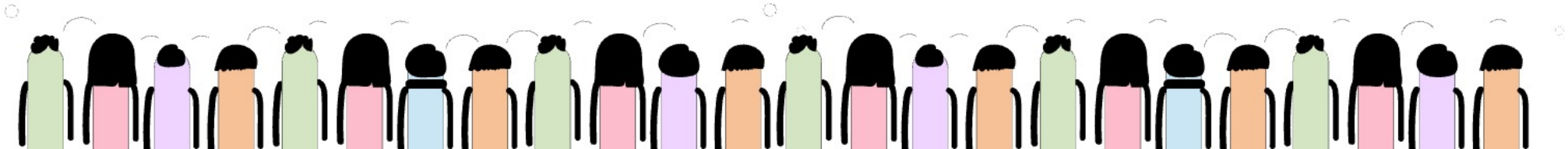
Applications depend on libc at runtime.



libc

Common libc Implementations

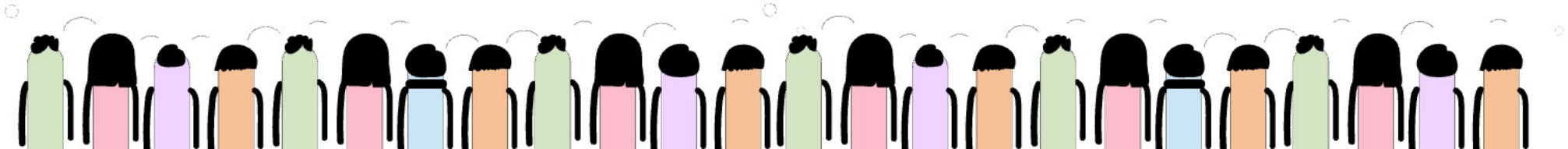
Library	Usage
glibc	Enterprise Linux
musl	Lightweight systems
uClibc	Embedded Linux



libc

Common Architectures

Architecture	Typical Usage
x86_64	Servers/desktops
ARM	Embedded/BMC/mobile
ARM64	Modern servers/mobile
RISC-V	Emerging embedded systems
PowerPC	Networking/legacy telecom



libc

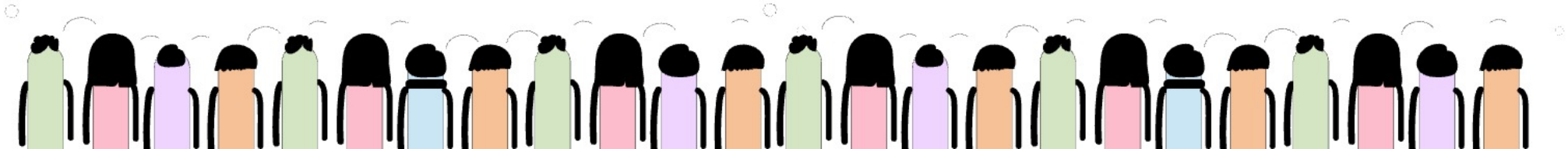
Real Production Example

Enterprise firmware workflow:

```
Developer Laptop (x86)
        ↓
ARM Cross Toolchain
        ↓
ARM Firmware Build
        ↓
BMC/iLO Hardware
        ↓
Remote Server Management
```

Final Real-World Understanding

Concept	Real Purpose
Native Compilation	Build and run on same architecture
Cross Compilation	Build for different target hardware
GCC	Compile source code
binutils	Link and analyze binaries
libc	Runtime library support
ARM	Efficient embedded processor architecture



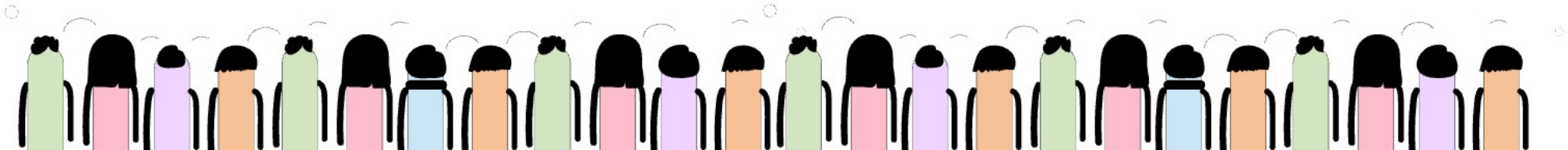
QEMU

1. What is QEMU?

Real Production Definition

QEMU (Quick Emulator) is an open-source machine emulator and virtualizer used to emulate complete hardware systems and CPU architectures in software.

(qemu.org ↗)



QEMU

QEMU vs Raspberry Pi

In embedded Linux and enterprise firmware development, engineers often need environments to:

- build firmware
- test kernels
- debug boot processes
- simulate hardware
- validate services
- test remote management systems

Two commonly used platforms are:

- QEMU (software emulation)
- Raspberry Pi (real ARM hardware)

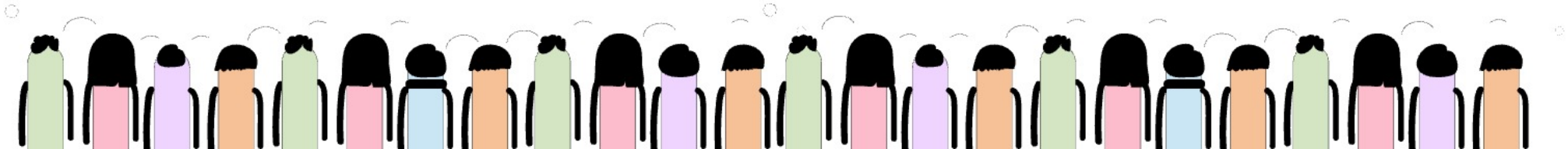
Both are widely used, but for different purposes.

What QEMU Does

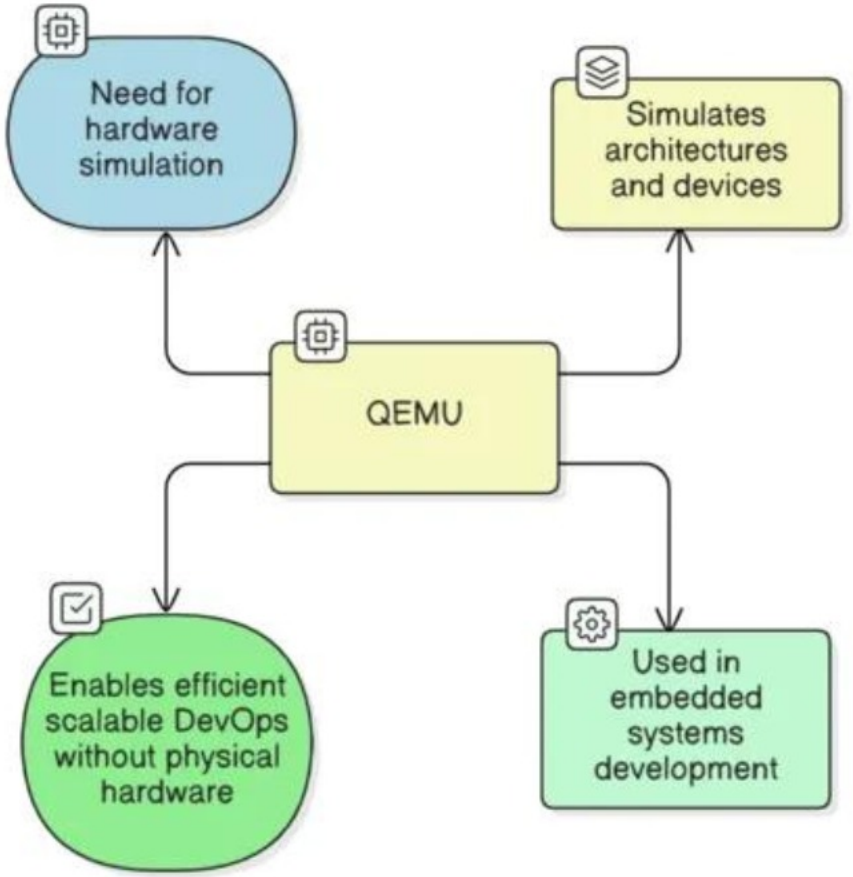
QEMU can emulate:

- ARM systems
- x86 servers
- RISC-V devices
- embedded boards
- networking hardware

without requiring physical hardware.



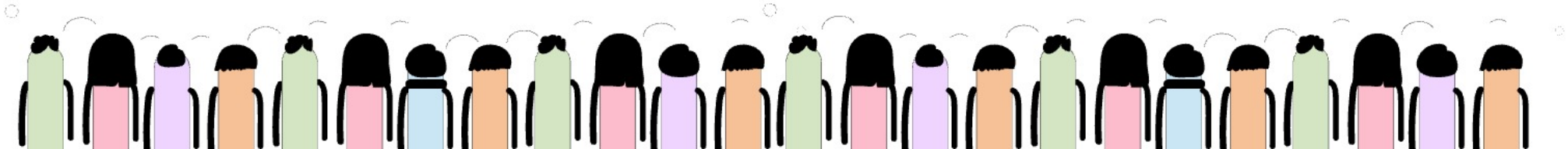
QEMU



Real Enterprise Usage

QEMU is heavily used in:

- embedded Linux development
- kernel development
- firmware validation
- CI/CD pipelines
- virtualization testing
- Yocto development
- OpenBMC testing



ARM

Example — ARM Firmware Emulation

```
Developer Laptop (x86)
        ↓
QEMU ARM Emulator
        ↓
Virtual ARM Hardware
        ↓
Embedded Linux Boots
```

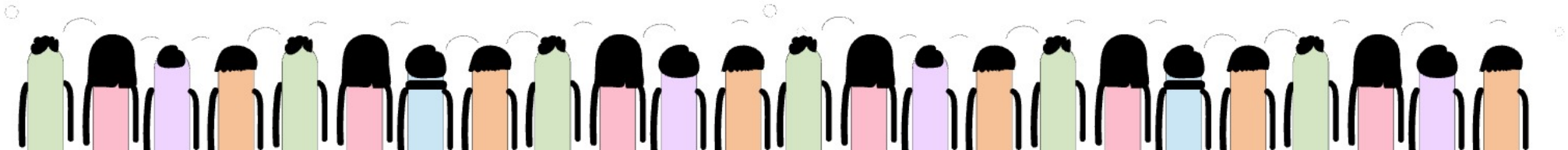
No physical ARM board required.

Real Production Example

Enterprise firmware teams use QEMU to test:

- Linux kernel boot
- U-Boot
- systemd services
- firmware recovery logic
- OpenBMC services

before deploying to physical hardware.

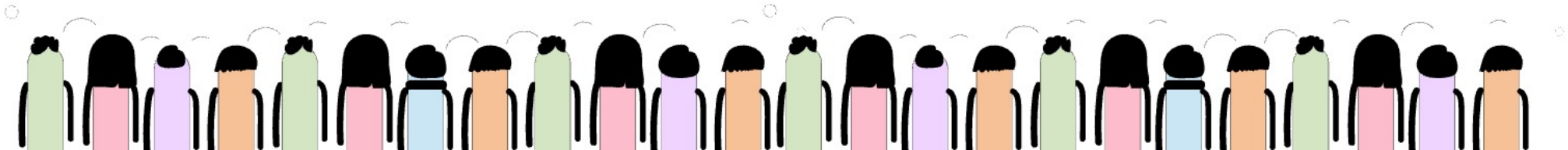


Raspberry

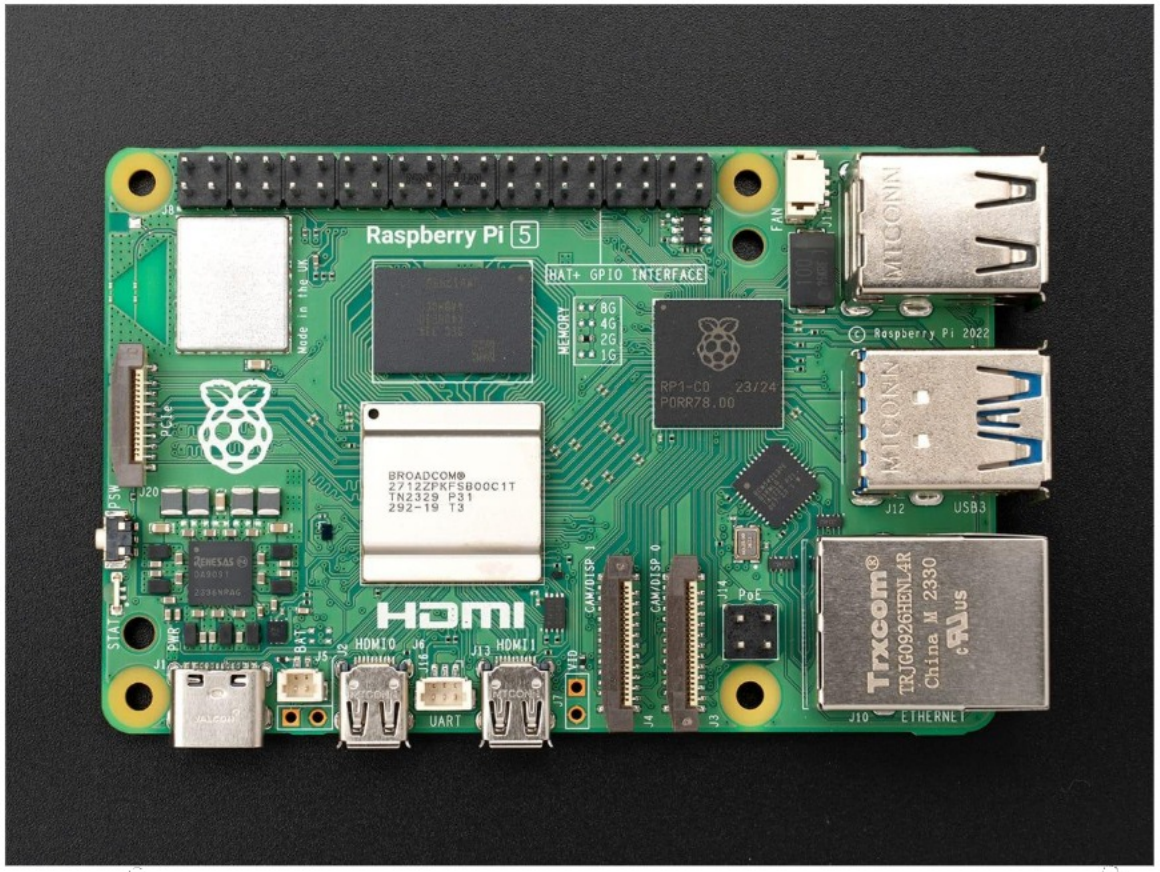
2. What is Raspberry Pi?

Real Production Definition

Raspberry Pi is a low-cost ARM-based single-board computer widely used for embedded Linux development, education, prototyping, and edge computing.



Raspberry

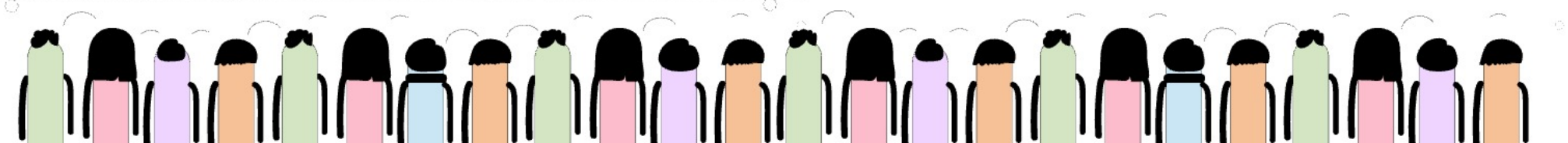


What Raspberry Pi Provides

Raspberry Pi provides real:

- ARM CPU
- GPIO hardware
- USB interfaces
- Ethernet
- HDMI
- storage interfaces

It is actual physical hardware.



Raspberry

Real Production Usage

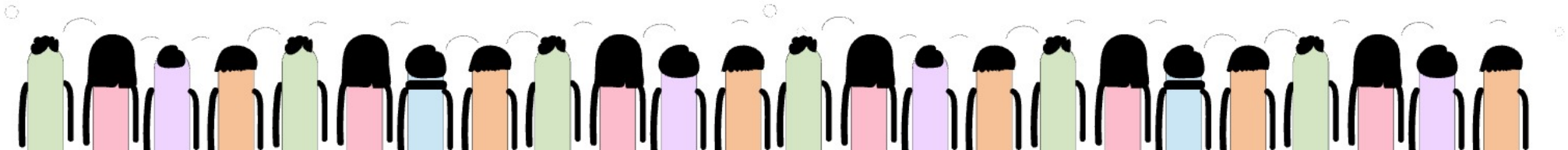
Raspberry Pi is commonly used for:

- embedded Linux training
- robotics
- IoT gateways
- edge computing
- sensor integration
- prototyping
- industrial automation

With QEMU:

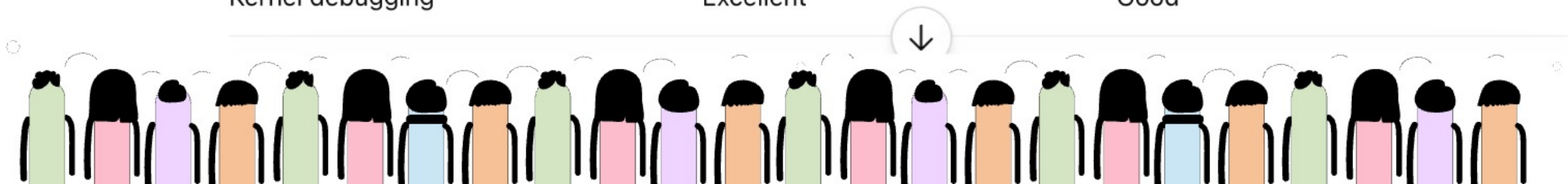
```
Modify Code
    ↓
Rebuild Firmware
    ↓
Boot Virtual System
    ↓
Debug Immediately
```

No hardware flashing required.



Raspberry

Area	QEMU	Raspberry Pi
Type	Software emulator	Physical ARM hardware
Hardware required	No	Yes
Cost	Low	Hardware purchase needed
Speed of testing	Very fast	Slower setup
Scalability	High	Limited
CI/CD integration	Excellent	Difficult
Hardware access	Virtualized	Real hardware
Sensor testing	Limited	Real peripherals
Kernel debugging	Excellent	Good



SSH

Real Production Definition

SSH (Secure Shell) is a secure remote access protocol used to manage Linux systems over networks.

Why SSH Is Critical

Enterprise embedded systems are often:

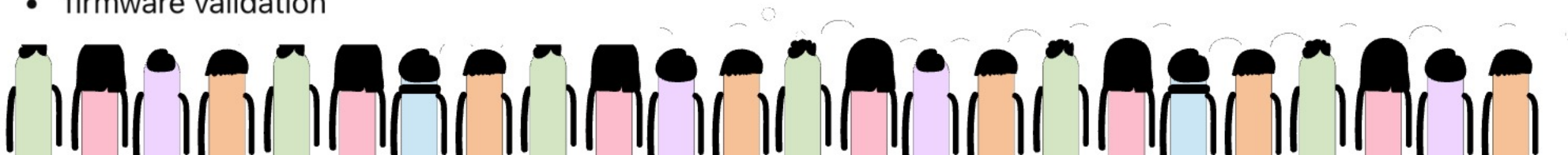
- headless
- remotely deployed
- without monitors/keyboards

So developers use SSH for:

- remote login
- debugging
- log analysis
- service management
- firmware validation

Example SSH Workflow

```
Developer Machine
        ↓
SSH
        ↓
Embedded Linux Device/QEMU
```



SSH

Real Production Commands

```bash
ssh root@192.168.1.10
```

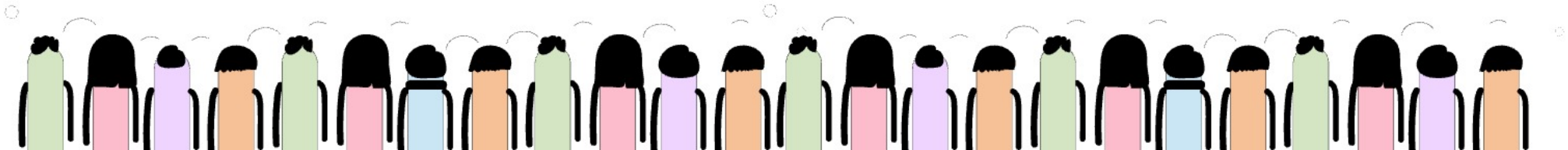
Then engineer can:

```bash
journalctl
systemctl status
dmesg
top
```

Where SSH Workflow Is Used

SSH workflows are standard in:

- datacenters
- BMC/iLO systems
- telecom infrastructure
- cloud servers
- embedded Linux devices
- OpenBMC platforms



System Thinking

Layer 1 — Application Thinking

“I wrote a feature.”

↓

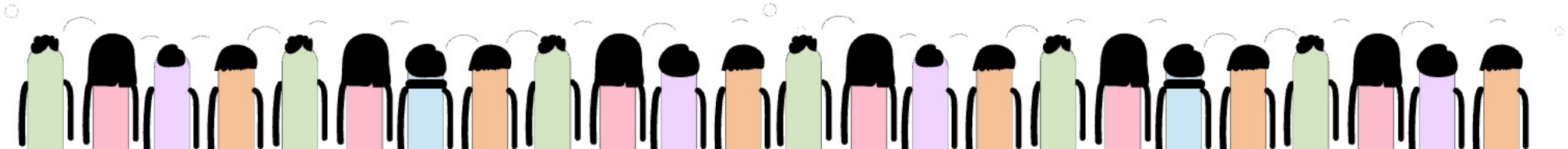
Layer 2 — System Thinking

“How entire OS behaves?”

↓

Layer 3 — Product Thinking

“How does device lifecycle work?”



System Thinking

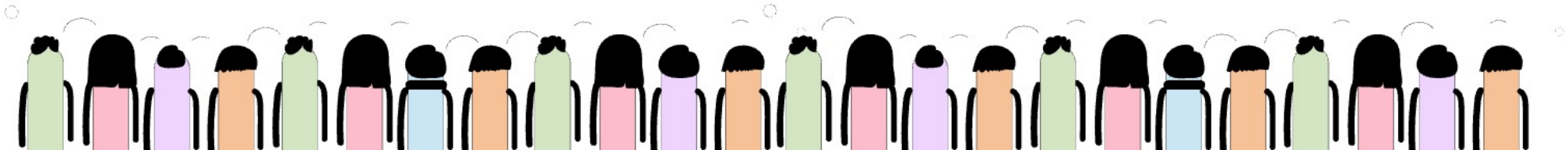
Layer 4 — Platform Thinking

“How do we build reusable firmware architecture?”

↓

Layer 5 — Enterprise Engineering Thinking

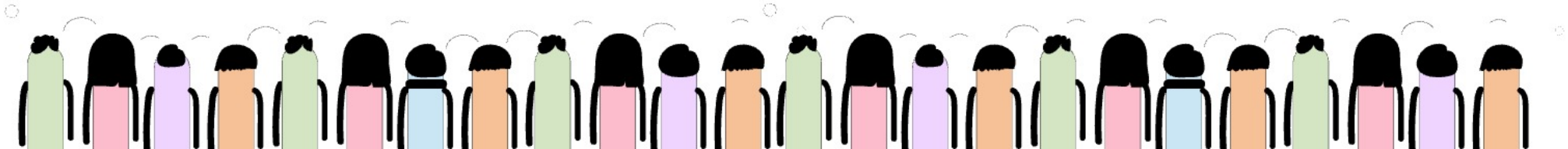
“How do we make millions of devices stable in production?”



Hewlett Packard Enterprise iLO

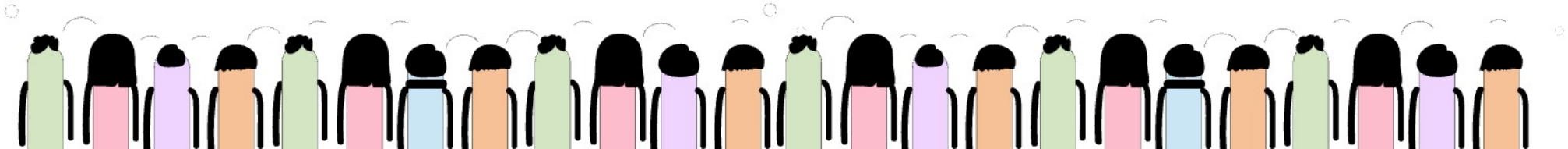
They are:

- firmware platform
- embedded Linux system
- remote datacenter controller
- hardware monitoring engine
- security platform
- enterprise management ecosystem



System Thinking

- What happens during boot?
- Why service starts late?
- Why memory usage high?
- Why watchdog reboot triggered?
- Why kernel panic happened?
- Why process hangs?

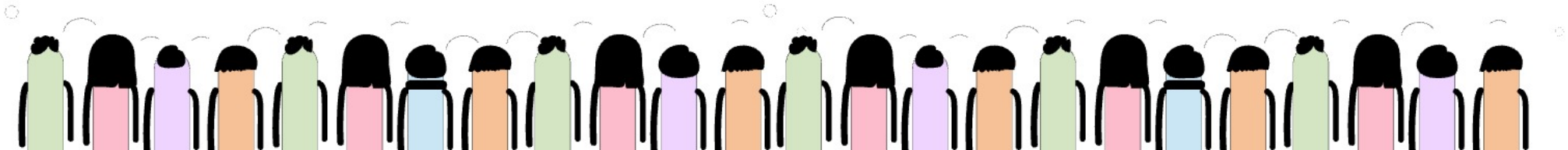


iLO

Integrated Lights-Out

It is a special **hardware + firmware management system** built by Hewlett Packard Enterprise for enterprise servers.

iLO = Tiny independent Linux computer inside server



iLO

SIMPLE UNDERSTANDING

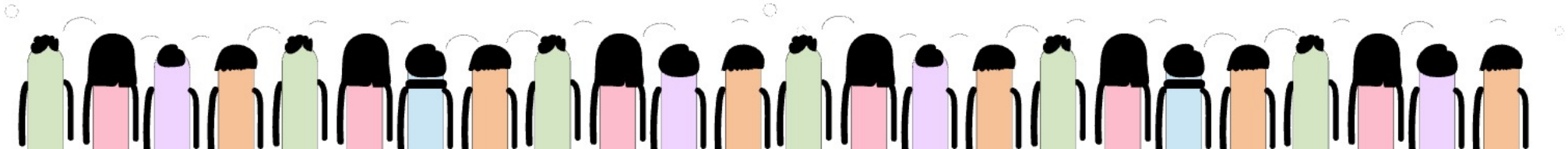
Think like this:

```
Normal Computer
   ↓
Needs OS running to manage system

But

Server with iLO
   ↓
Can be controlled EVEN IF main OS is dead
```

That is the magic.

REAL ARCHITECTURE

```
+-----------------------------------+
| Enterprise Server                 |
|                                   |
| Main CPU + Main OS                |
| Linux / Windows / VMware          |
|                                   |
|-----------------------------------|
|                                   |
| iLO / BMC                         |
| ARM Processor                     |
| Embedded Linux Firmware           |
| Dedicated Network Access          |
|                                   |
+-----------------------------------+
```



iLO

Monitor
Manage
Recover
Control
Protect
Update
Diagnose

WHY IT EXISTS

Datacenter servers may be:

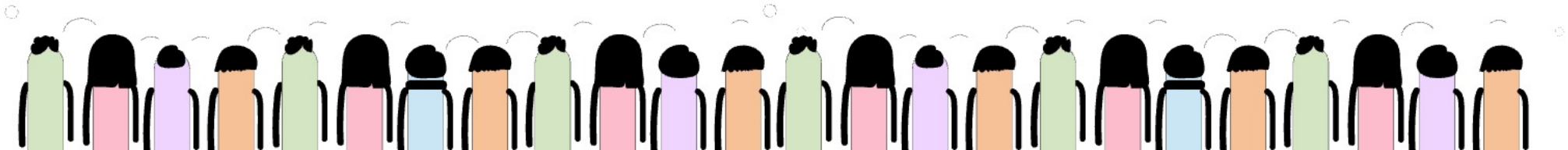
- in another country
- inside locked racks
- running 24x7 critical workloads

Physical access impossible.

So companies need:

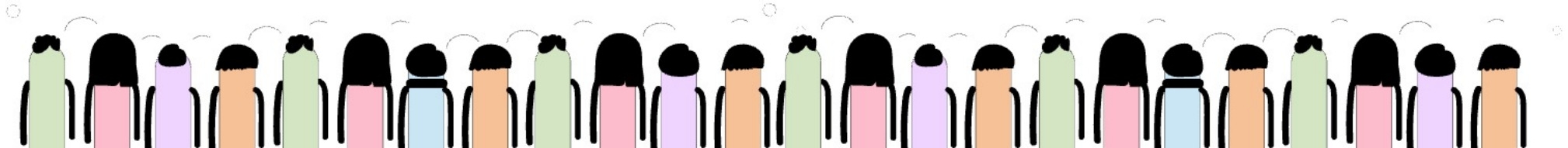
- remote control
- remote recovery
- hardware monitoring

That is what iLO provides.



ILO Features

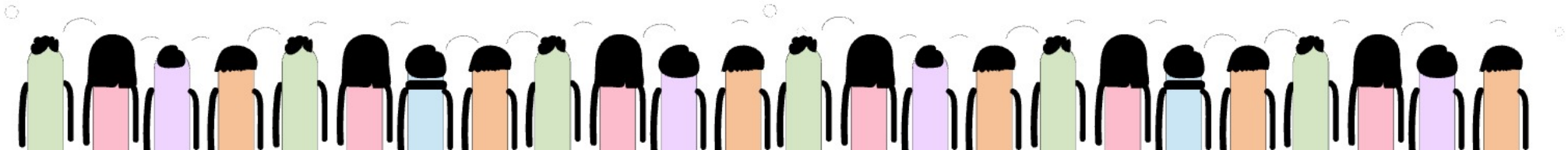
Feature	Purpose
Power ON/OFF	Remote reboot
View hardware logs	Diagnose failures
Monitor temperature	Prevent overheating
Fan control	Thermal management
Remote console	View server screen remotely
Mount ISO remotely	Install OS remotely
Firmware updates	Upgrade server firmware
Sensor monitoring	Health checks
SSH/API access	Automation



iLO

```
Server Power ON
      ↓
iLO Starts First
      ↓
Hardware Initialization
      ↓
Main Server CPU Starts
      ↓
Main OS Boots
```

Area	Real Work
Boot flow	U-Boot → Kernel → systemd
Memory	RAM leaks
CPU usage	Service optimization
Drivers	Sensor communication
Networking	Dedicated management NIC
Security	Access control
Logging	Persistent diagnostics



Server

SCENARIO

A customer has:

- 500 HPE servers in datacenter
- Running banking workloads
- Server room temperature suddenly increases
- CPU temperature rises
- Fan control service behaves incorrectly

Now entire product engineering chain activates.

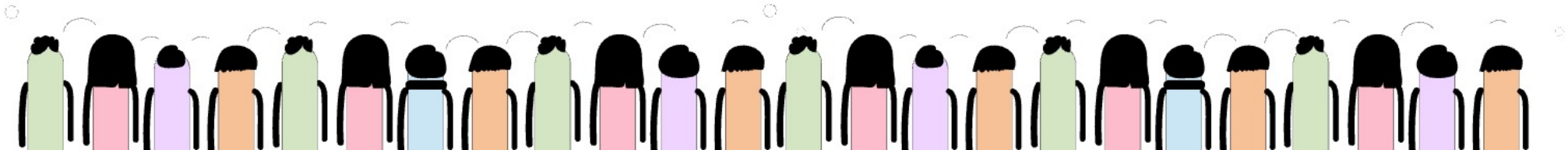
STEP 1 — HARDWARE LEVEL

Physical Reality

Inside server:

- CPU temperature sensors
- Fan RPM sensors
- Power sensors
- Thermal controllers

These connect to BMC/iLO board.



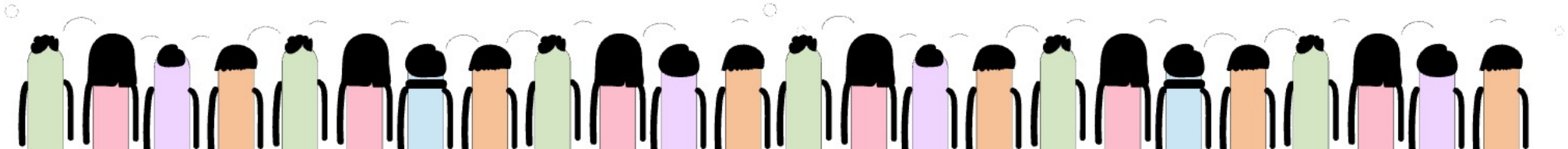
Server

WHAT USER EXPERIENCES

Customer sees:

Problem	Impact
Fan speed frozen	Temperature rises
CPU throttling	Slow applications
Thermal shutdown	Server goes OFF
Banking app downtime	Huge business loss

Now issue becomes enterprise-critical.



Server

Engineer SSH into iLO Linux.

Checks:

```bash
systemctl status fan-control
```

Finds:

```
Service failed
Segmentation fault
```

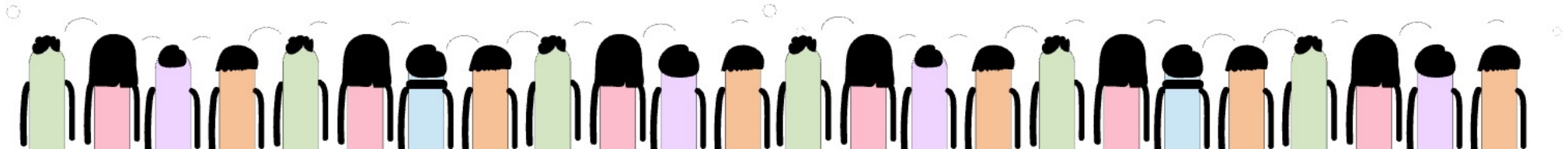
STEP 5 — LOG ANALYSIS

Checks:

```bash
journalctl -u fan-control
```

Sees:

```
Temperature parsing failed
NULL pointer access
```

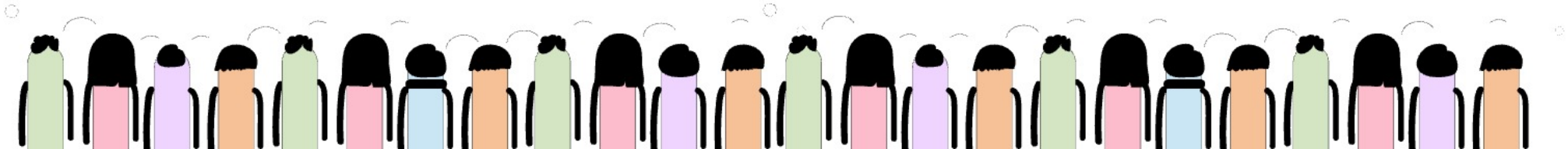


Server

REAL BOOT FLOW

```
Server Power ON
        ↓
iLO Starts First
        ↓
Hardware Initialization
        ↓
Main Server CPU Starts
        ↓
Main OS Boots
```

iLO monitors everything continuously.



iLO

A fan-control service crashes.

System-thinking engineer investigates:

- journalctl logs
- kernel logs
- process dependencies
- startup order
- memory corruption
- IPC failures

Not just application code.

What They Build

Firmware Update Engine

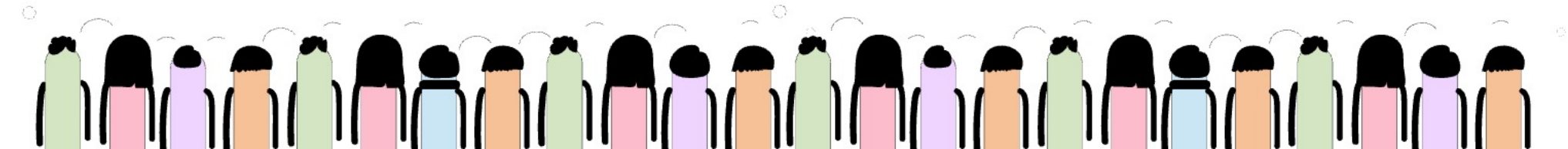
- dual image support
- rollback recovery
- corruption detection
- signed image validation

Remote Management

Even when server OS dead:

- reboot server
- view hardware logs
- mount ISO remotely
- reinstall OS remotely

This is why BMC/iLO is critical.



iLO

Layer 1 — Application Thinking

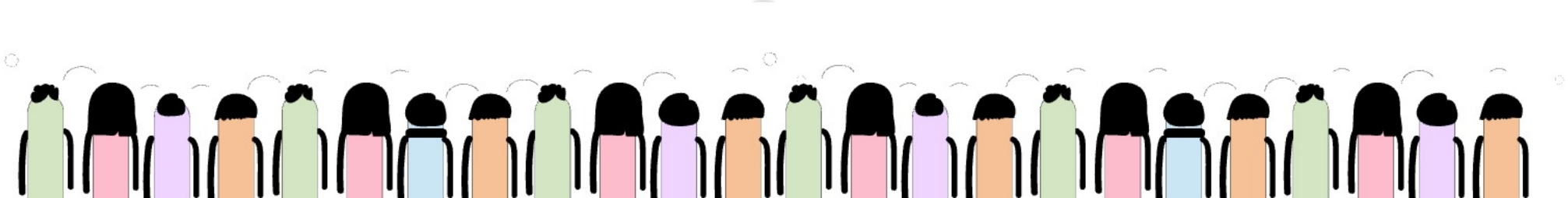
“I fixed fan service bug.”

Layer 2 — System Thinking

“How service interacts with Linux + sensors + kernel?”

Layer 3 — Product Thinking

“How do we prevent hardware overheating?”



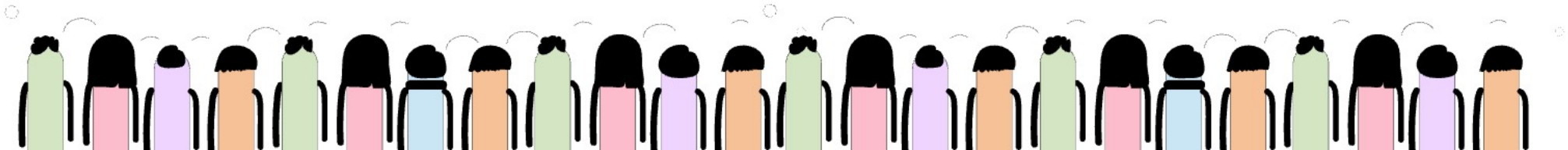
iLO

Layer 4 — Platform Thinking

“How make reusable thermal framework for all servers?”

Layer 5 — Enterprise Thinking

“How do we protect 1 million servers globally from thermal failure?”



Remote Service

SECOND EXAMPLE — REMOTE MANAGEMENT

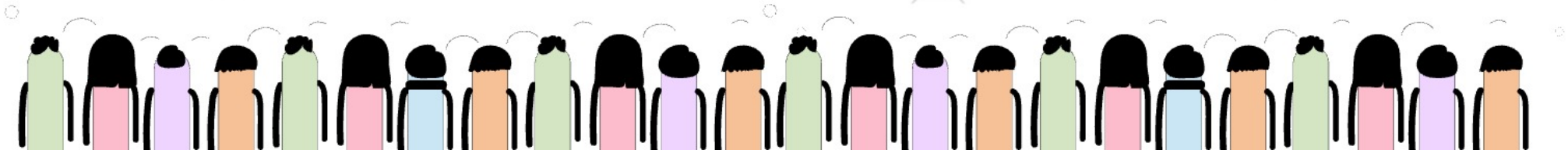
Now imagine server OS completely dead.

Windows/Linux crashed.

Normal remote desktop impossible.

But iLO still works because:

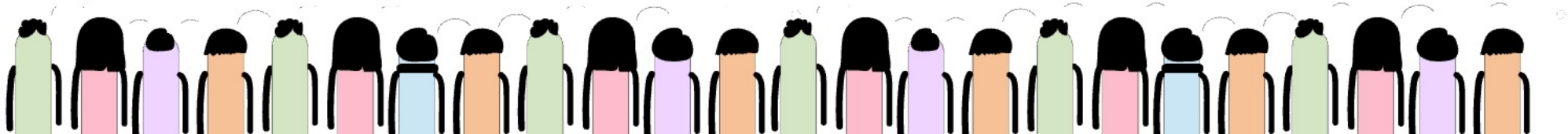
```
Main Server CPU → DEAD
Main OS → DEAD

But

BMC ARM Processor → STILL RUNNING
Embedded Linux → STILL RUNNING
iLO → STILL ACCESSIBLE
```



Day1 Flow

BIG PICTURE FLOW

```
Your Ubuntu Laptop (x86 CPU)
            ↓
Cross Compiler creates ARM binary
            ↓
QEMU emulates ARM CPU
            ↓
ARM binary executes on Ubuntu
            ↓
You test embedded firmware/software
WITHOUT real ARM board
```



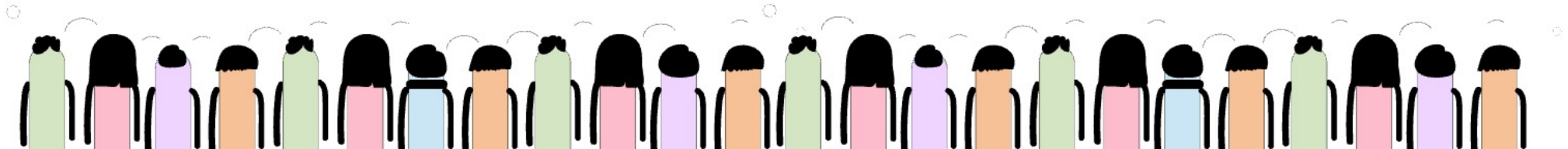
Day1 Flow

COMPLETE END-TO-END INDUSTRY FLOW

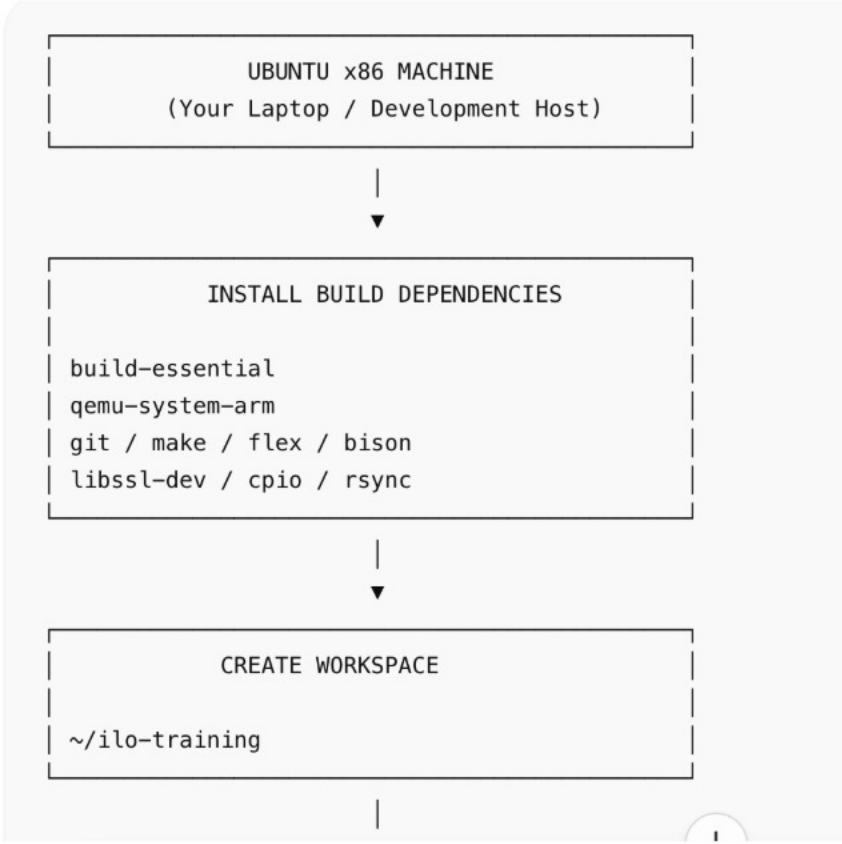
```
Developer writes C code
          ↓
Cross compiler generates ARM binary
          ↓
Linux creates ELF executable
          ↓
QEMU emulates ARM CPU
          ↓
ARM runtime libraries loaded
          ↓
Program executes on x86 Ubuntu
```

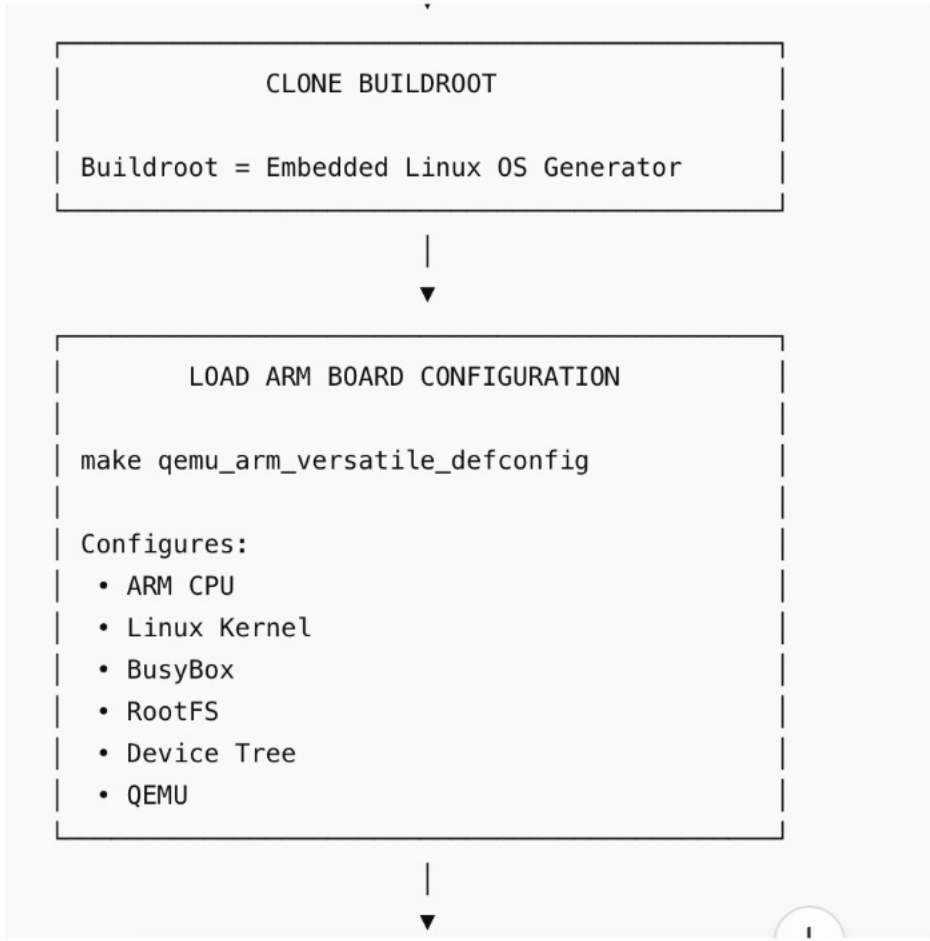
INTERNAL EXECUTION FLOW

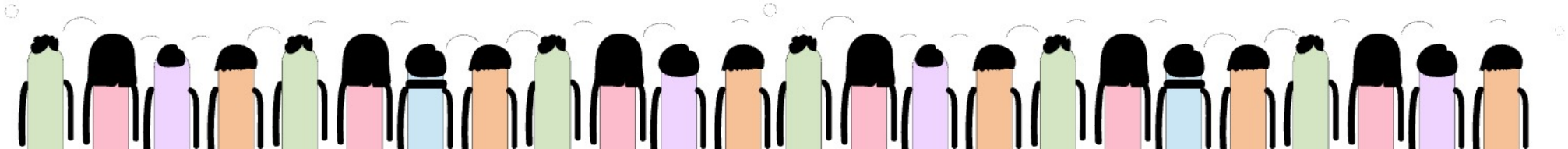
```
ARM Binary
     ↓
QEMU reads ARM instructions
     ↓
Translates ARM → x86
     ↓
Loads ARM shared libraries
     ↓
Executes program
     ↓
Prints output
```



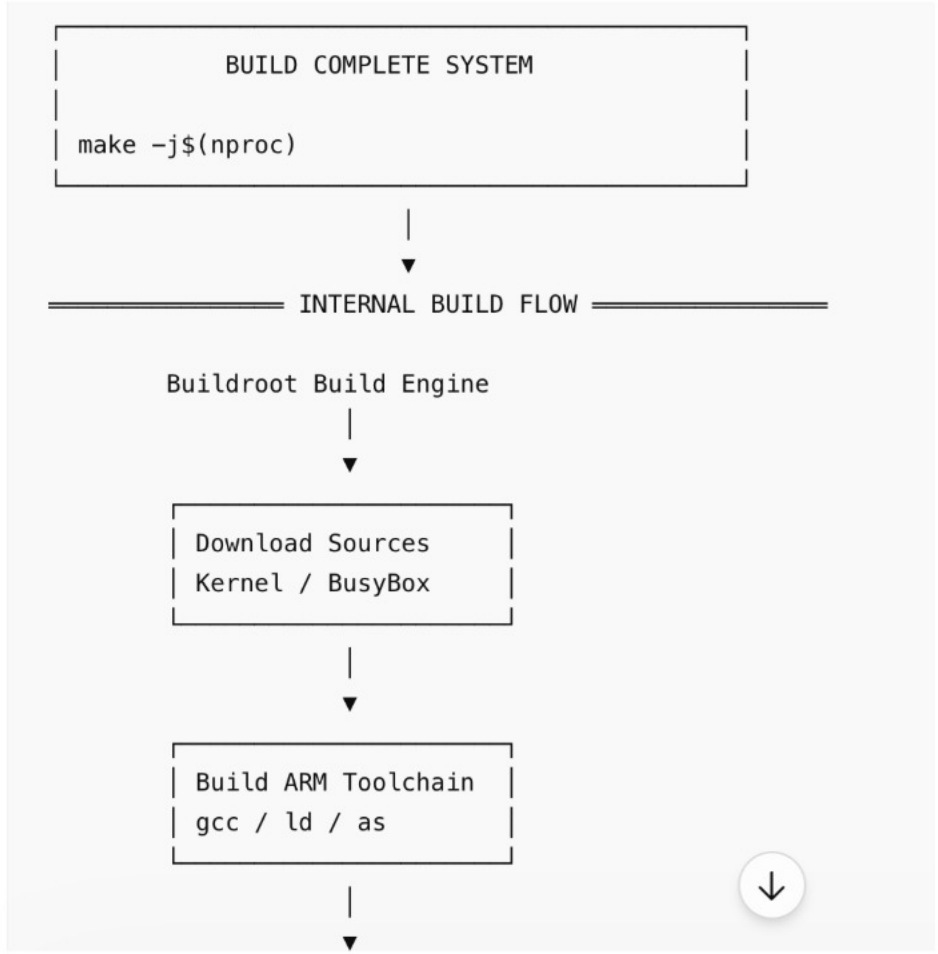
Day2 Flow

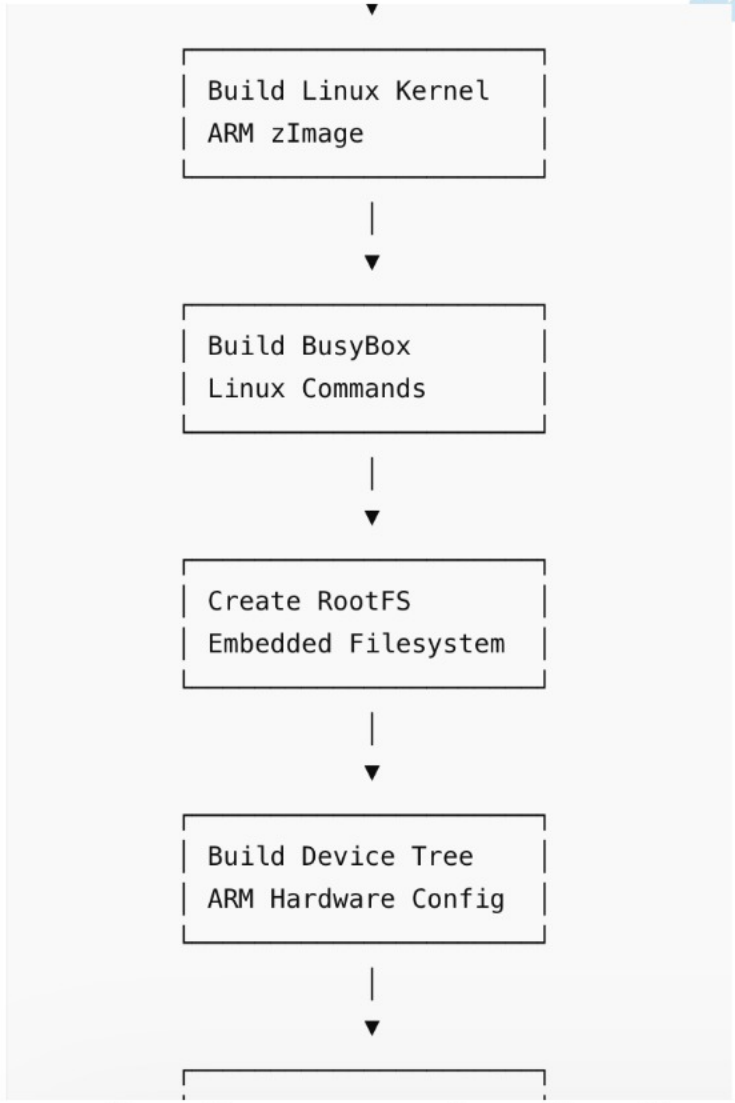

UBUNTU x86 MACHINE
(Your Laptop / Development Host)
INSTALL BUILD DEPENDENCIES
build-essential
qemu-system-arm
git / make / flex / bison
libssl-dev / cpio / rsync
CREATE WORKSPACE
~/ilo-training

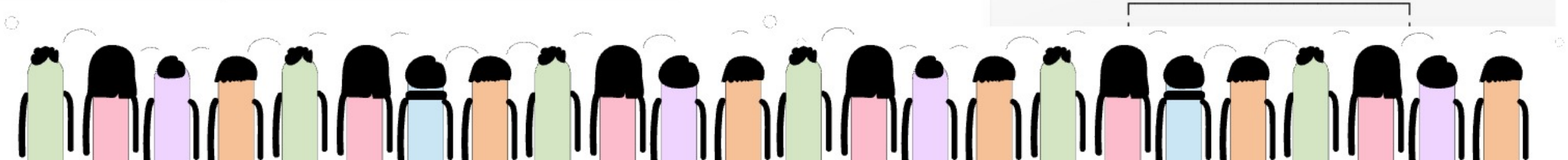

CLONE BUILDROOT
Buildroot = Embedded Linux OS Generator
LOAD ARM BOARD CONFIGURATION
make qemu_arm_versatile_defconfig
Configures:
• ARM CPU
• Linux Kernel
• BusyBox
• RootFS
• Device Tree
• QEMU



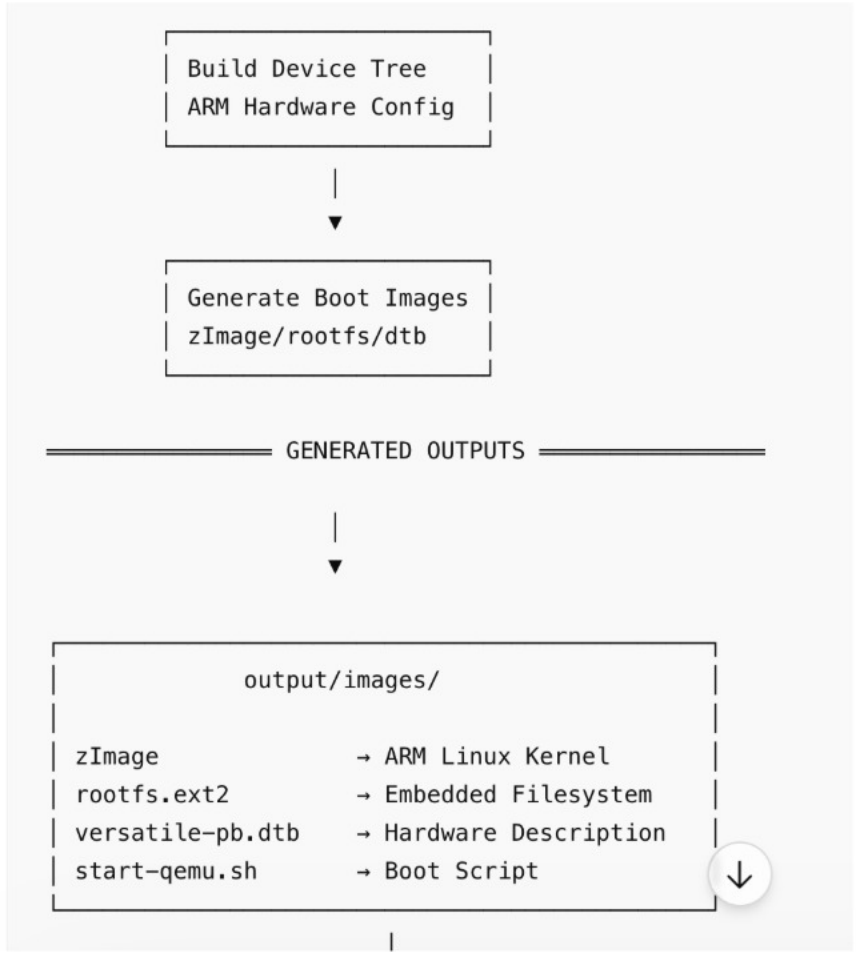
Day2 Flow

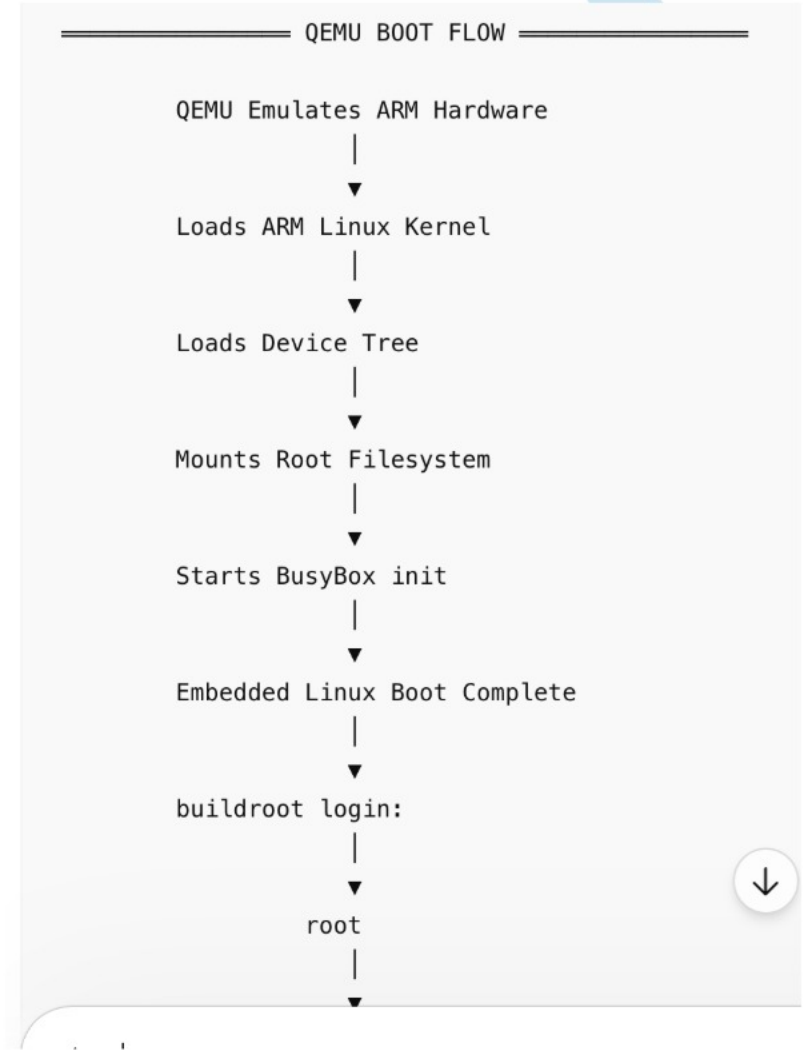

BUILD COMPLETE SYSTEM
make -j$(nproc)
INTERNAL BUILD FLOW
Buildroot Build Engine
Download Sources
Kernel / BusyBox
Build ARM Toolchain
gcc / ld / as

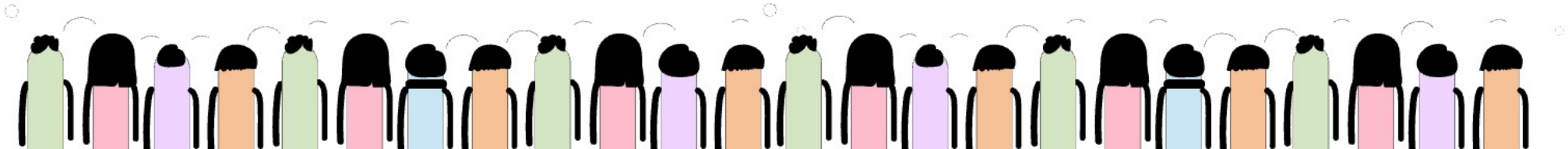

Build Linux Kernel
ARM zImage
Build BusyBox
Linux Commands
Create RootFS
Embedded Filesystem
Build Device Tree
ARM Hardware Config



Day2 Flow


Build Device Tree
ARM Hardware Config
Generate Boot Images
zImage/rootfs/dtb
GENERATED OUTPUTS
output/images/
zImage → ARM Linux Kernel
rootfs.ext2 → Embedded Filesystem
versatile-pb.dtb → Hardware Description
start-qemu.sh → Boot Script

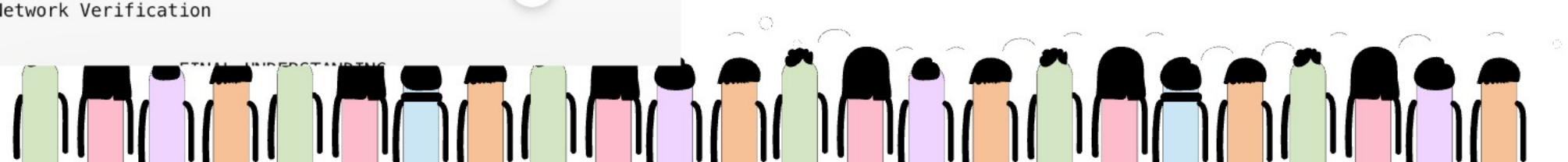

QEMU BOOT FLOW
QEMU Emulates ARM Hardware
Loads ARM Linux Kernel
Loads Device Tree
Mounts Root Filesystem
Starts BusyBox init
Embedded Linux Boot Complete
buildroot login:
root



Day2 Flow

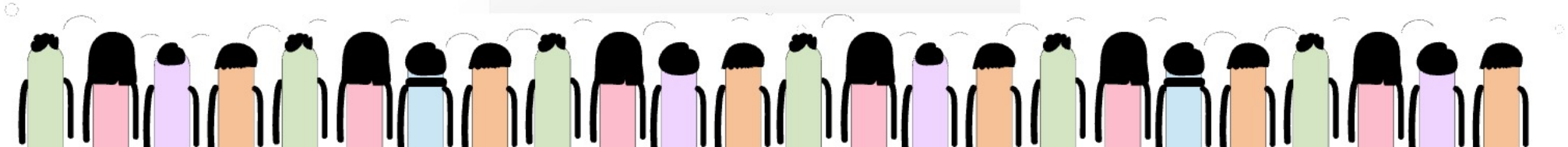
```
═══════════════ SYSTEM VERIFICATION ═══════════════

uname -a
    ↓
Kernel Verification

cat /proc/cpuinfo
    ↓
ARM CPU Verification

ps
    ↓
Running Processes

mount
    ↓
Filesystem Verification

free -h
    ↓
Memory Verification

ifconfig
    ↓
Network Verification
```

```
Source Code
      ↓
Buildroot
      ↓
ARM Toolchain
      ↓
Linux Kernel
      ↓
BusyBox
      ↓
Root Filesystem
      ↓
Boot Images
      ↓
QEMU ARM Emulator
      ↓
Embedded Linux Running
```



Day3 Flow

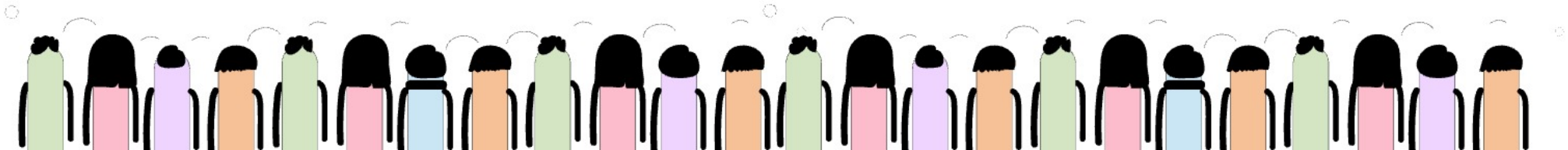
```
Ubuntu Host
      ↓
Buildroot
      ↓
ARM Toolchain
      ↓
Linux Kernel
      ↓
Root Filesystem
      ↓
U-Boot Bootloader
      ↓
QEMU ARM Hardware
      ↓
Linux Boot
      ↓
systemd Init
      ↓
Mini iLO Service
      ↓
journald Logging
      ↓
Firmware-style Embedded Linux Platform
```



Day3 Flow

```
Power ON
   ↓
Bootloader
   ↓
Kernel
   ↓
Drivers
   ↓
RootFS
   ↓
systemd/init
   ↓
BusyBox/Services
   ↓
IPC/D-Bus
   ↓
Firmware Applications
```

```
Power ON
   ↓
CPU Reset Vector
   ↓
BootROM
   ↓
Bootloader (U-Boot)
   ↓
Linux Kernel
   ↓
Device Tree (DTB)
   ↓
initramfs/rootfs
   ↓
PID 1 init/systemd
   ↓
Services Start
   ↓
Embedded Firmware Running
```



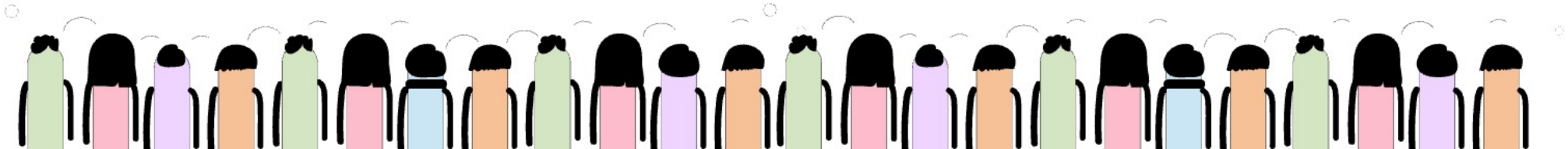
Yocto

Suppose You Want To Build Firmware For ARM Board

You need:

```
Linux kernel
BusyBox
Libraries
/bin
/etc
drivers
boot files
```

Manually doing this is huge pain.



Yocto

Without Yocto

You manually do:

```
Bash

download kernel
download busybox
compile kernel
compile busybox
copy files
create rootfs
create image
```

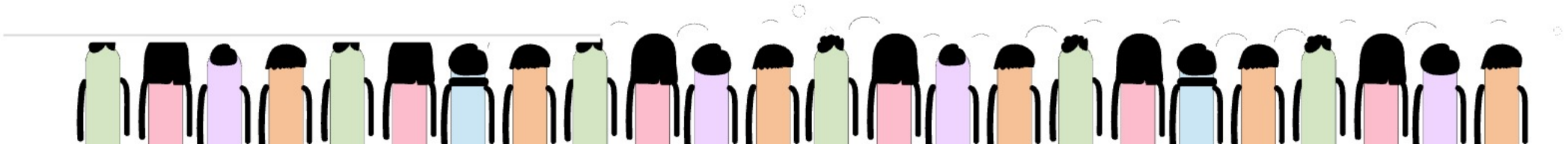
100s of manual steps.

Yocto Does This Automatically

You simply say:

```
Bash

bitbake core-image-minimal
```

Then Yocto internally does all work.



Yocto

REAL SIMPLE FLOW

Step 1 — Download Things

Yocto downloads:

```
Linux kernel
BusyBox
OpenSSL
systemd
```

like internet downloads.

Step 2 — Open / Prepare Them

Example:

```
linux.tar.gz
```

Yocto extracts it.

Same like:

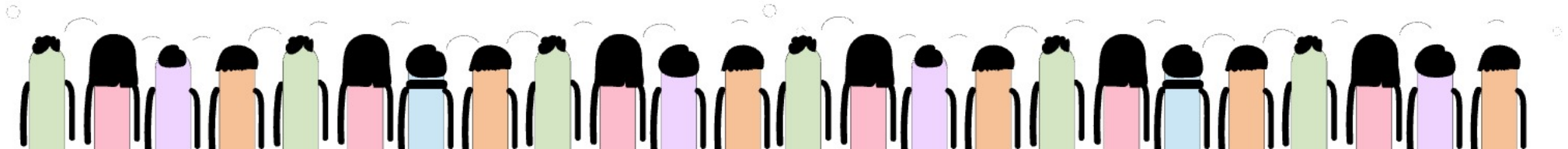
```
‹/› Bash

tar -xf linux.tar.gz
```

Step 3 — Configure

Yocto sets options.

Example:

```
enable ARM
enable networking
enable ext4
```



Yocto

Step 4 — Compile

Very important.

Yocto runs:

```bash
arm-linux-gcc
```

to convert:

```
C code → ARM binary code
```

Example:

```
ls.c → ls binary
```

Step 5 — Create Linux Filesystem

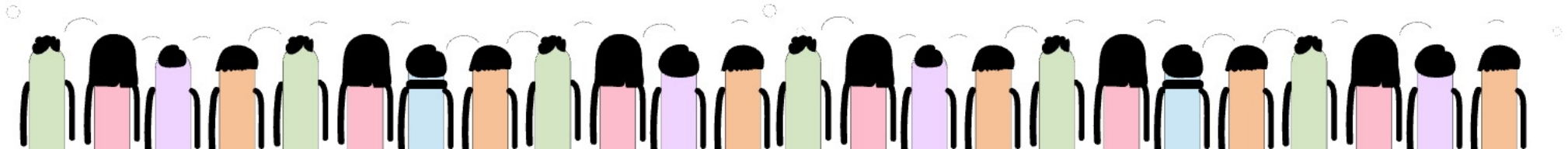
Yocto now creates folders like:

```
/bin
/lib
/etc
/usr
```

and copies compiled binaries there.

Example:

```
/bin/busybox
/lib/libc.so
```



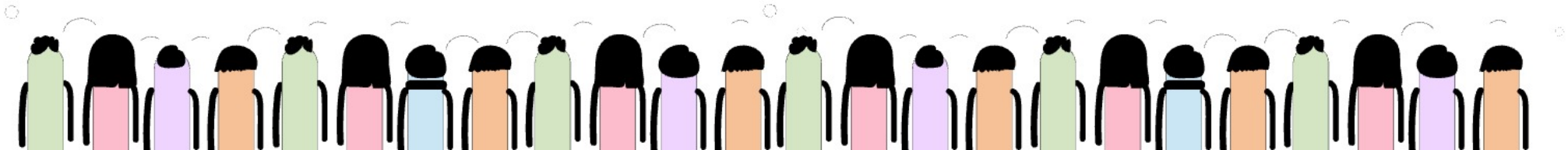
Yocto

Step 6 — Create Final Image

Now Yocto packs everything into:

```
sdcard.img
rootfs.ext4
uImage
```

This becomes bootable firmware.



Yocto

REAL HUMAN ANALOGY

Yocto Is Like:

```
automatic factory
```

INPUT

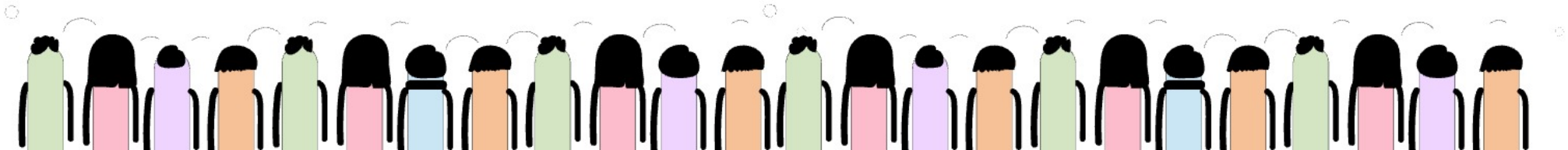
```
source code
configs
recipes
```

MACHINE WORK

```
download
extract
compile
copy
organize
pack
```

OUTPUT

```
bootable Linux firmware
```



Yocto

Why Yocto Better Than Makefile

Makefile

Usually builds:

```
one software
```

Yocto

Builds:

```
entire Linux operating system
```

Where BitBake Comes

BitBake is simply:

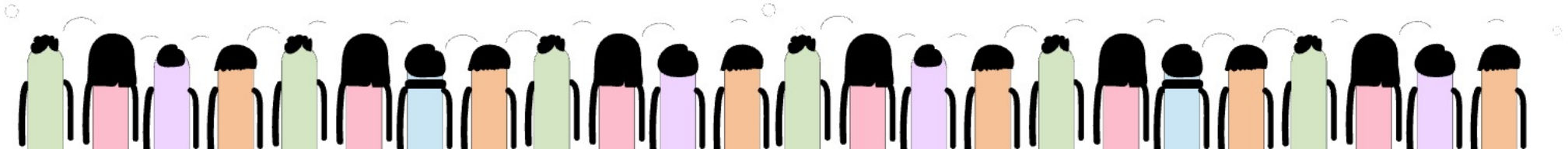
```
the manager controlling all steps
```

Where Recipes (.bb) Come

Recipe says:

```
where source exists
how to compile
where to copy
```

like cooking instructions.



Yocto

In Yocto

Task looks like:

```
do_compile()
```

This is basically:

```
build function
```

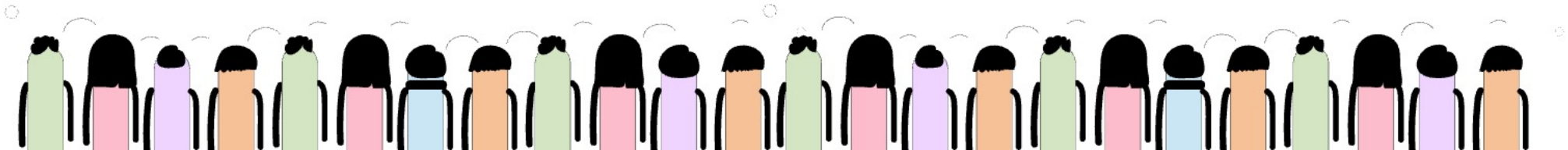
executed by BitBake.

Real Internal Idea

BitBake sees:

```
do_fetch
do_unpack
do_compile
do_install
```

like ordered functions/tasks.

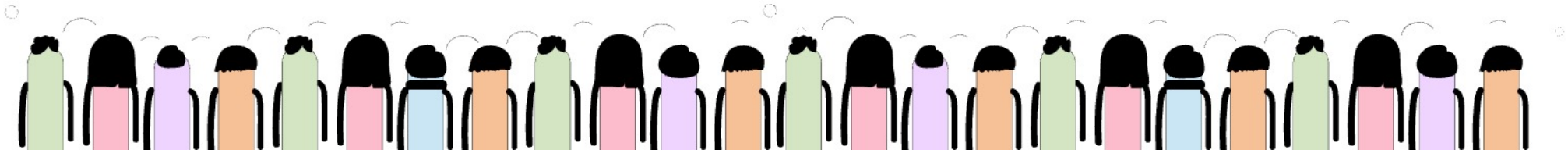


Yocto

Internally BitBake Executes

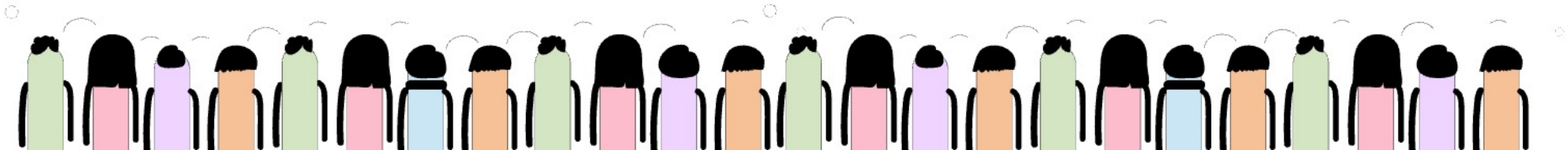
```
Task 1
 ↓
Task 2
 ↓
Task 3
```

in dependency order.



Yocto

Programming	Yocto
Function	Task
Function call	Task execution
Program flow	Build pipeline



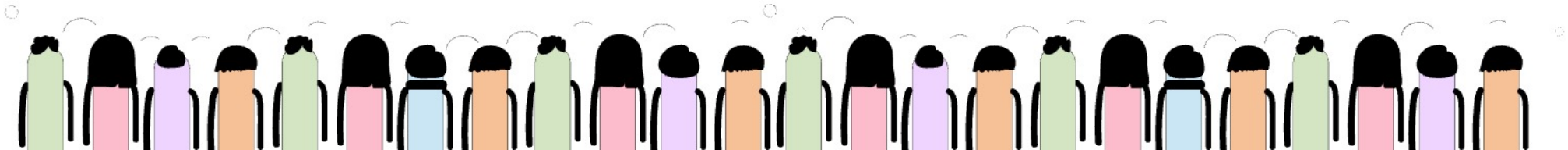
Yocto

```
do_fetch()
    ↓
do_unpack()
    ↓
do_configure()
    ↓
do_compile()
    ↓
do_install()
```

Each is basically:

```
special build function
```

```
Source Code
   ↓
Fetch
   ↓
Extract
   ↓
Modify
   ↓
Configure
   ↓
Compile
   ↓
Install into rootfs
   ↓
Package
   ↓
Assemble filesystem
   ↓
Generate firmware image
```



Yocto

FIRST UNDERSTAND

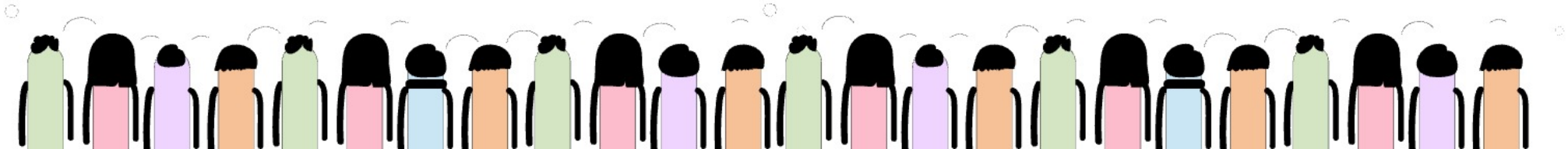
Yocto internally works like:

```
Directed Dependency Graph (DAG)
```

NOT simple linear execution.

REAL TASK GRAPH

```
do_fetch()
     ↓
do_unpack()
     ↓
do_patch()
     ↓
do_configure()
     ↓
do_compile()
     ↓
do_install()
     ↓
do_package()
     ↓
do_rootfs()
     ↓
do_image()
```



Yocto

1. `do_fetch()`

Real Functionality

Python

```
download_source()
```

Dependency

Needs:

```
SRC_URI
network access
download manager
```

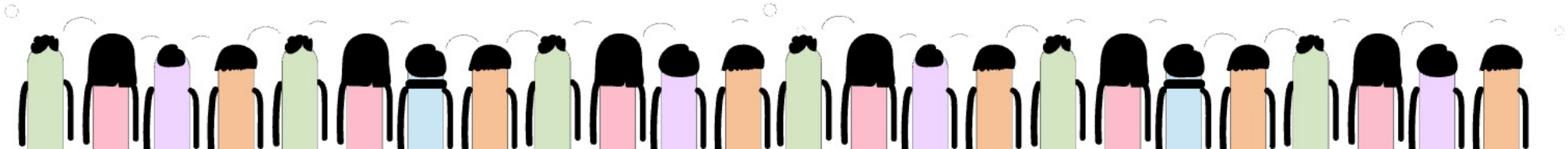
Output

```
downloads/package.tar.gz
```

Graph Connection

```
do_unpack depends on do_fetch
```

because unpack cannot happen without source.



Yocto

2. `do_unpack()`

Real Functionality

Python

```
extract_archive()
```

Internally

Uses:

Bash

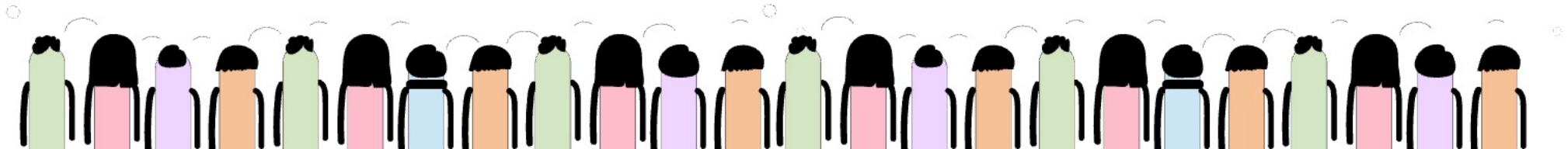
```
tar
unzip
git checkout
```

Dependency

```
needs downloaded source
```

Output

```
source directory
```



Yocto

3. `do_patch()`

Real Functionality

Python

```
apply_patch()
```

Dependency

Needs extracted source.

Output

```
modified source tree
```

4. `do_configure()`

Real Functionality

Python

```
generate_build_configuration()
```

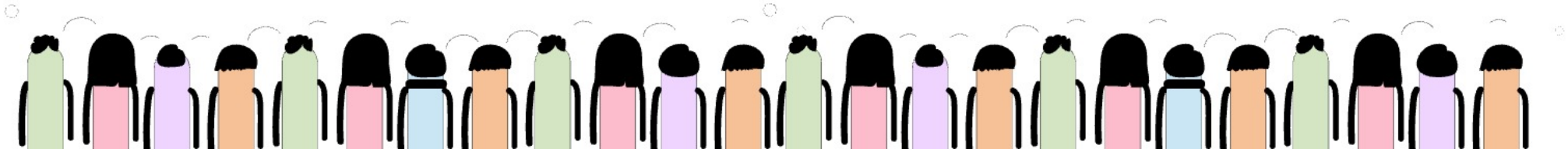
Internally Runs

Bash

```
./configure
cmake
make menuconfig
```

Dependency

Needs final patched source.



Yocto

5. `do_compile()`

MOST IMPORTANT

Real Functionality

Python

```python
cross_compile()
```

Internally

Runs:

Bash

```bash
arm-linux-gcc
```

6. `do_install()`

Real Functionality

Python

```python
copy_to_rootfs_staging()
```

Dependency

Needs compiled binaries.

Output

```
image/bin
image/lib
image/etc
```

9. `do_image()`

FINAL FUNCTION

Real Functionality

Python

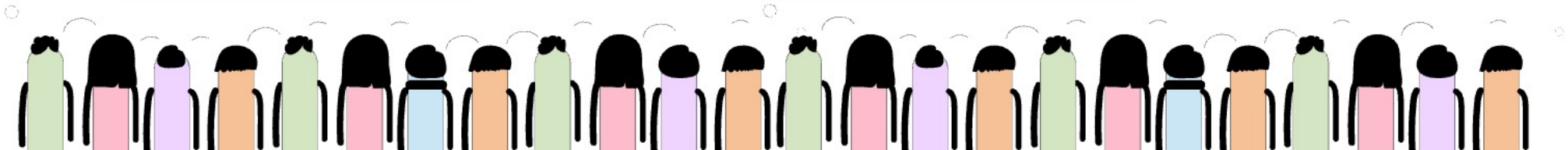
```python
generate_bootable_firmware_image()
```

Dependency

Needs complete rootfs.

Output

```
sdcard.img
rootfs.ext4
wic image
```



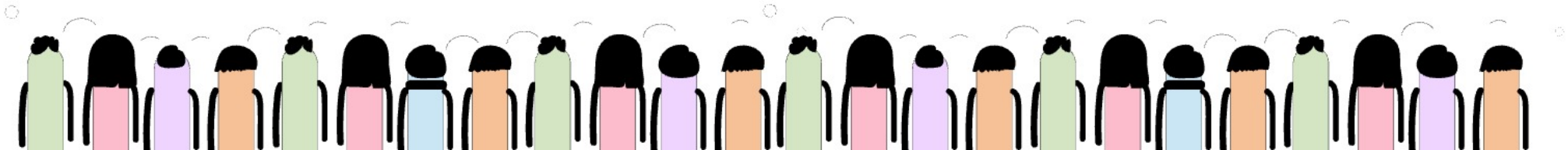
Yocto -Step1

Bash

```
${CXX} ${CXXFLAGS} sensor-monitor.cpp -o sensor-monitor
```

Flow:

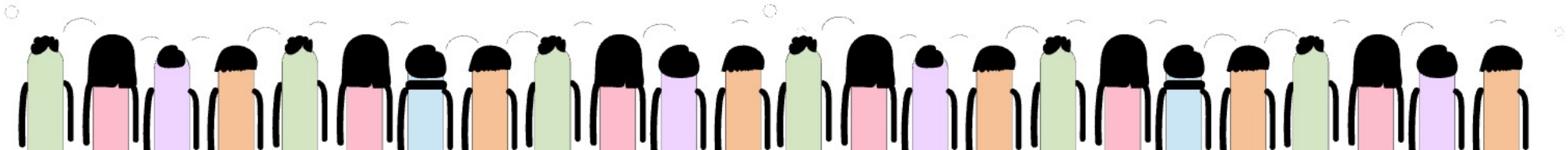
```
step1
   ↓
C++ compiler starts
   ↓
reads sensor-monitor.cpp
   ↓
compiles source code
   ↓
creates executable binary
   ↓
sensor-monitor
```



Yocto

COMPLETE REAL FLOW

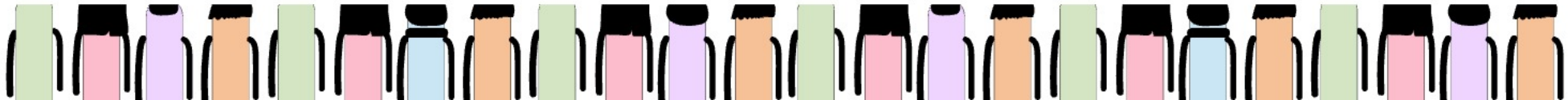
```
sensor-monitor.cpp
        ↓
${CXX}
        ↓
${CXXFLAGS}
        ↓
compile object code
        ↓
${LDFLAGS}
        ↓
link libraries
        ↓
generate executable
        ↓
sensor-monitor
```



Yocto -step2

YOUR STEP 2 UNDERSTANDING

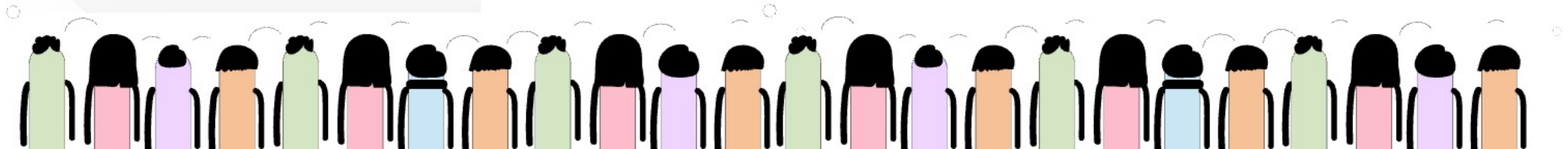
```
compile executable
        ↓
copy executable into rootfs staging area
        ↓
copy systemd service file
        ↓
package creation
```



Yocto -Step2

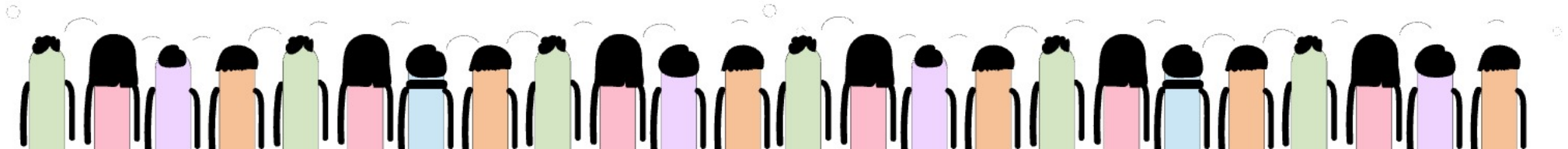
```
sensor-monitor.cpp
        ↓
compile
        ↓
sensor-monitor executable
        ↓
do_install()
        ↓
copy into rootfs staging
        ↓
/usr/bin/sensor-monitor

sensor-monitor.service
        ↓
copy into
/lib/systemd/system/
        ↓
package generation
        ↓
rootfs image
        ↓
QEMU boot
        ↓
systemd starts service
```

Variable	Meaning
`${D}`	rootfs staging directory
`${bindir}`	`/usr/bin`
`${systemd_system_unitdir}`	`/lib/systemd/system`
`${WORKDIR}`	recipe temp work area



Yocto -Step2

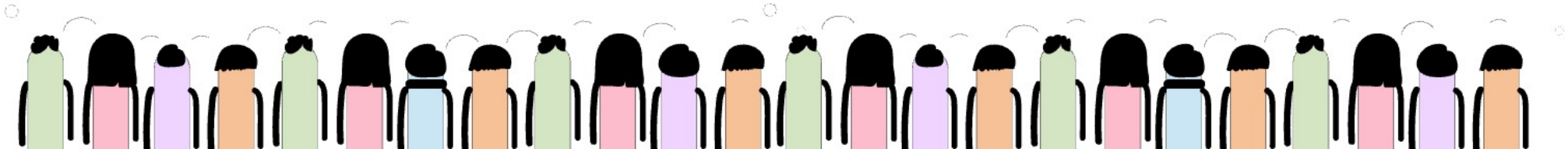
```
do_compile()
      ↓
binary generated

do_install()
      ↓
copy into ${D}

do_package()
      ↓
create rpm/deb/ipk

do_rootfs()
      ↓
assemble filesystem

do_image()
      ↓
generate firmware image
```



Yocto -Step3

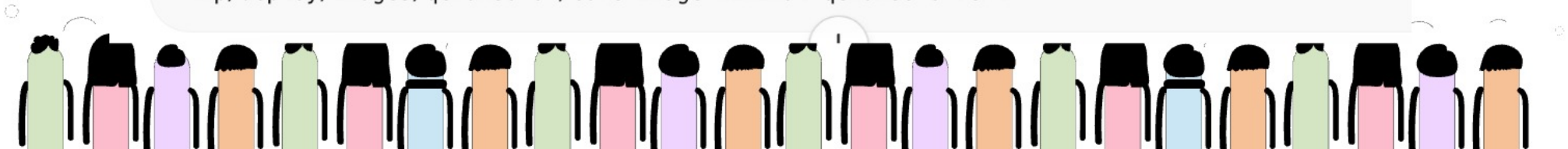
WHAT THESE FOLDERS MEAN

Folder	Purpose
all-poky-linux	architecture independent packages
core2-64-poky-linux	target binaries/packages
qemux86_64-poky-linux	machine-specific builds
x86_64-linux	native host tools



Yocto -Step3

```
sensor-monitor.cpp
        ↓
do_compile()
        ↓
tmp/work/core2-64-poky-linux/sensor-monitor/1.0-r0/sensor-monitor

do_install()
        ↓
image/usr/bin/sensor-monitor

do_package()
        ↓
packages-split/sensor-monitor/usr/bin/sensor-monitor

do_rootfs()
        ↓
tmp/work/qemux86_64-poky-linux/core-image-minimal/1.0-r0/rootfs/usr/bin/sensor-monitor

do_image()
        ↓
tmp/deploy/images/qemux86-64/core-image-minimal-qemux86-64.ext4
```



Yocto -Step3

GO INSIDE

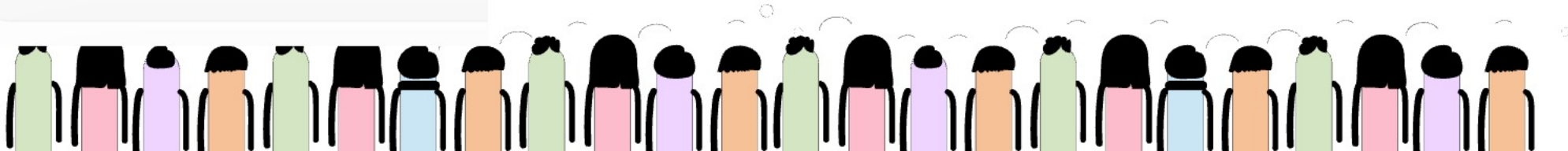
```bash
cd core2-64-poky-linux
```

LIST PACKAGES

```bash
ls
```

You may see:

```
sensor-monitor
fan-control
watchdog-monitor
host-state-manager
```



Yocto -Step3

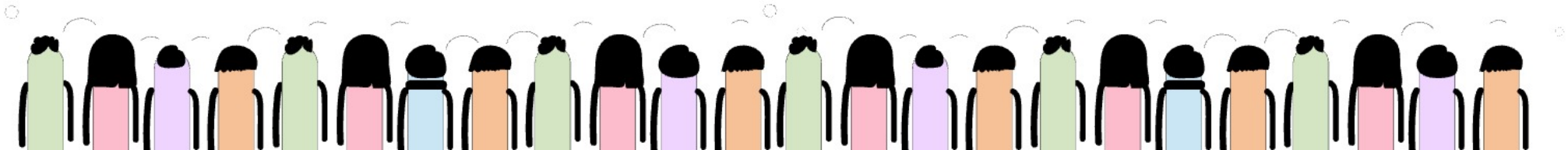
You are inside the correct location already.

Current path:

```
/home/venkateswara/poky/build/tmp/work/core2-64-poky-linux/sensor-monitor/1.0-r0
```

Now your question is:

```
where final files are present?
```



Yocto -Step3

1. GENERATED EXECUTABLE

You already see:

```
sensor-monitor
```

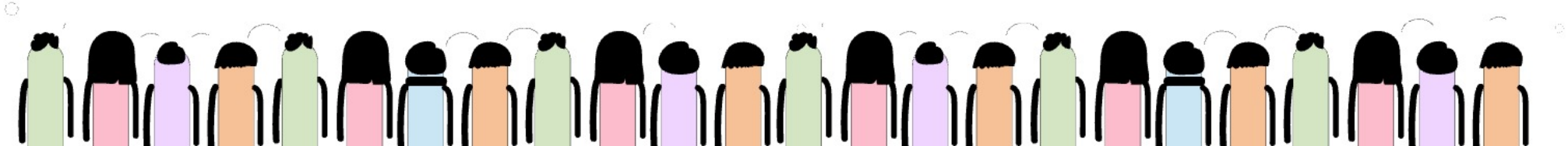
This file is the compiled executable binary.

Generated during:

```
do_compile()
```

Location:

```
~/poky/build/tmp/work/core2-64-poky-linux/sensor-monitor/1.0-r0/sensor-monitor
```



Yocto -Step3

3. FINAL PACKAGE CONTENTS

Go inside:

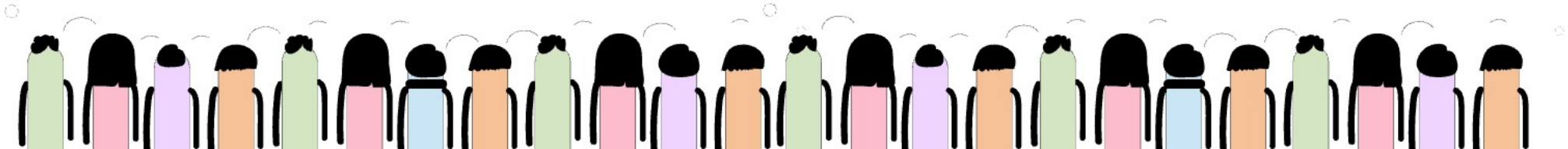
```bash
cd ../packages-split
```

Then:

```bash
find .
```

You will see:

```
sensor-monitor/usr/bin/sensor-monitor

sensor-monitor/lib/systemd/system/sensor-monitor.service
```



Yocto -Step3

4. FINAL RPM/IPK PACKAGE

Go inside:

```bash
cd ../deploy-rpms
```

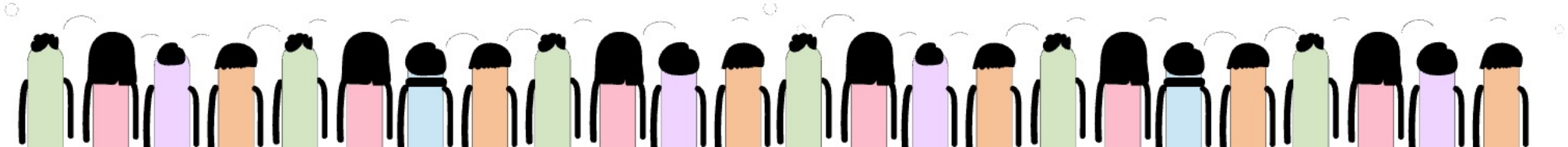
or check:

```bash
find . -name "*.rpm"
```

You may see:

```
sensor-monitor-1.0-r0.core2_64.rpm
```

This is actual installable package.



Yocto -Step3

5. FINAL IMAGE LOCATION

Go here:

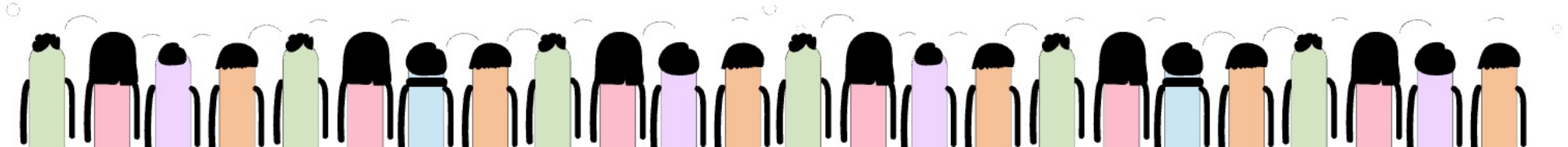
```bash
cd ~/venkateswara/poky/build/tmp/deploy/images/qemux86-64
```

Then:

```bash
ls
```

You will see:

```
core-image-minimal-qemux86-64.ext4

bzImage
```



Yocto -Step3

```
sensor-monitor.cpp
        ↓
do_compile()
        ↓
sensor-monitor executable
        ↓
do_install()
        ↓
image/usr/bin/sensor-monitor
        ↓
do_package()
        ↓
packages-split/sensor-monitor/
        ↓
do_rootfs()
        ↓
core-image-minimal rootfs
        ↓
do_image()
        ↓
final ext4 image
```

Yocto -Step4

FINAL ROOTFS CONTAINS

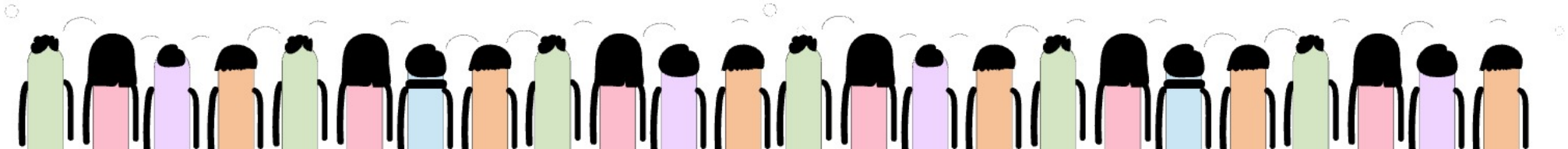
Inside:

```
core-image-minimal rootfs
```

you will have:

```
/usr/bin/sensor-monitor

/lib/systemd/system/sensor-monitor.service
```

Correct understanding.



Yocto -Step4

STEP 2

Inside:

```
tmp/work/core2-64-poky-linux/
```

Yocto stores:

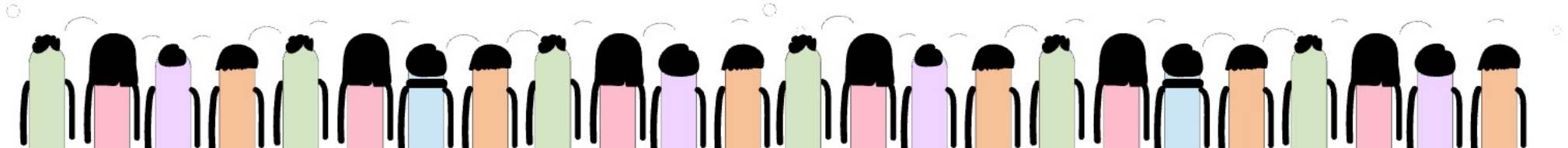
```
per-package build workspace
```

Example:

```
~/poky/build/tmp/work/core2-64-poky-linux/sensor-monitor/1.0-r0/
```

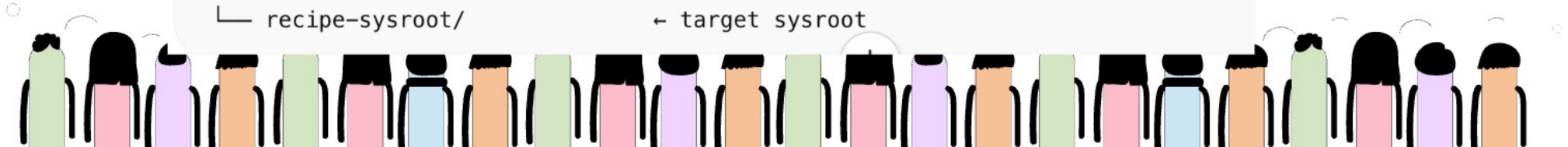
This contains:

- source files
- compile outputs
- temp logs
- staged install files
- package split files



Yocto -Step4

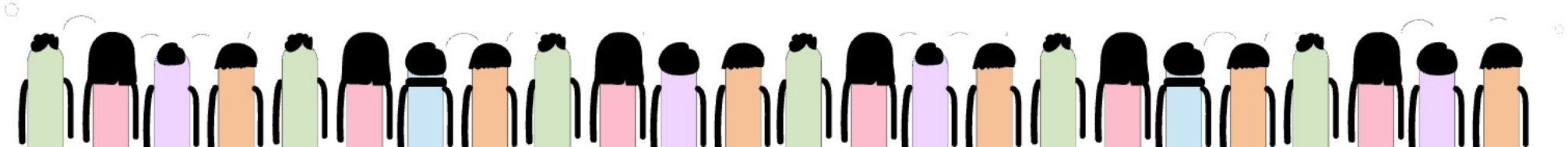
```
~/poky/build/tmp/work/core2-64-poky-linux/sensor-monitor/1.0-r0/

├── sensor-monitor                ← compiled executable
├── sensor-monitor.cpp
├── sensor-monitor.service
│
├── temp/                         ← build logs
│     ├── log.do_compile
│     ├── log.do_install
│     └── log.do_package
│
├── image/                        ← staged rootfs area
│     ├── usr/bin/sensor-monitor
│     └── lib/systemd/system/sensor-monitor.service
│
├── packages-split/               ← final package contents
│     └── sensor-monitor/
│          ├── usr/bin/sensor-monitor
│          └── lib/systemd/system/sensor-monitor.service
│
├── package/                      ← packaging metadata
├── pkgdata/                      ← package database
└── recipe-sysroot/               ← target sysroot
```



Yocto -Step4

WHAT EACH AREA STORES

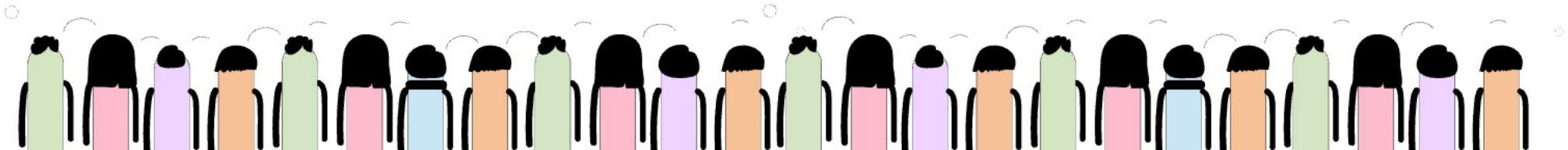
Folder	Stores
temp	build/install/package logs
image	temporary rootfs install files
packages-split	final package filesystem
package	packaging metadata
recipe-sysroot	target headers/libs
sensor-monitor	compiled executable



Yocto -Step4

WHAT THESE MEAN

Directive	Meaning
Requires=	hard dependency
After=	startup order only



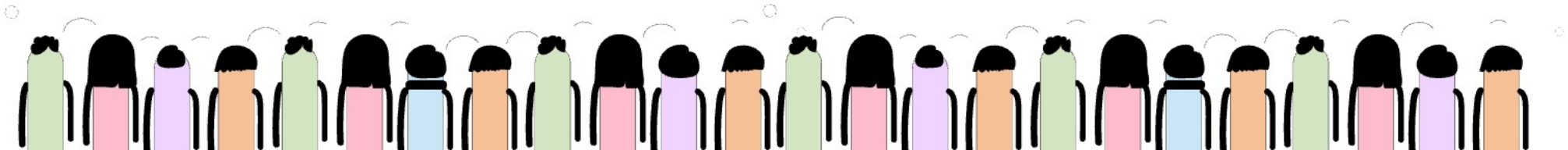
Yocto -Step4

WHAT YOU SEE IN QEMU

```bash
systemctl status fan-control
```

Output may show:

```
Dependency failed for Fan Control Service
```

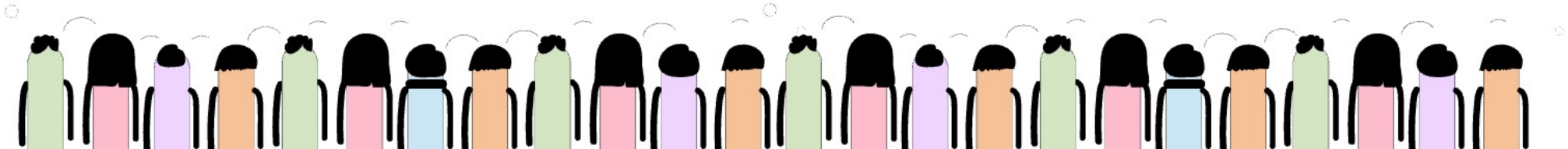


Yocto Clean

STEP 3 — CLEAN EVERYTHING IMPORTANT

VERY IMPORTANT.

```bash
bitbake -c clean sensor-monitor

bitbake -c cleanall sensor-monitor

rm -rf tmp/work/core2-64-poky-linux/sensor-monitor

rm -rf tmp/work/qemux86_64-poky-linux/core-image-minimal
```



Yocto Rebuild

STEP 4 — REBUILD PACKAGE FIRST

Bash

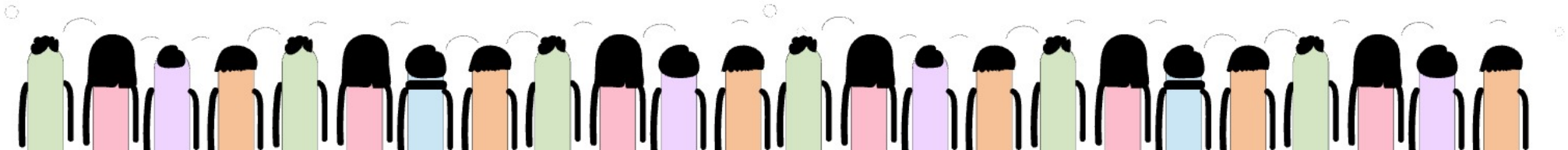
```
bitbake sensor-monitor
```

MUST succeed FIRST.

STEP 6 — BUILD IMAGE

Bash

```
bitbake core-image-minimal
```



Real Time

1. KERNEL PANIC DEBUGGING

What Is Kernel Panic?

Kernel crashes completely.

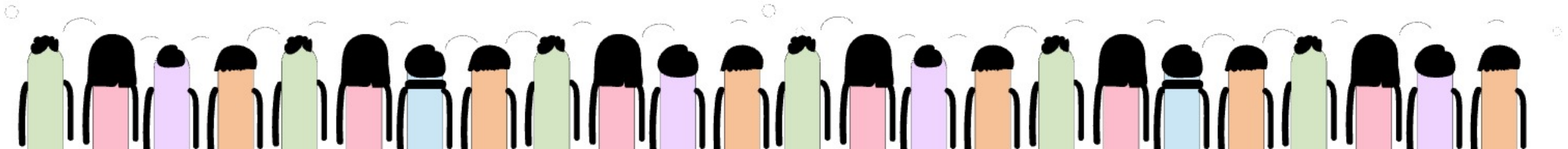
Equivalent of:

```
OS fatal crash
```

System usually freezes/reboots.

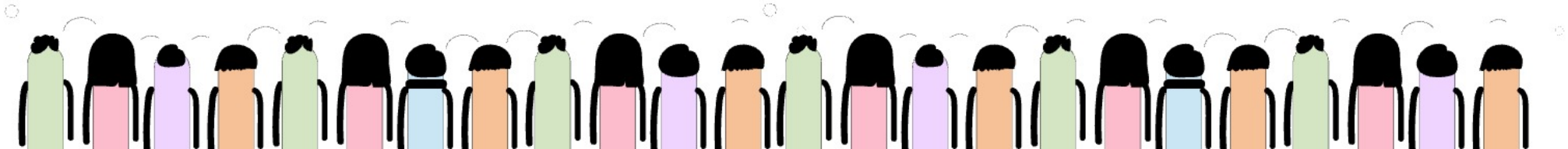
REAL Production Log

```
Kernel panic - not syncing: VFS: Unable to mount root fs
CPU: 0 PID: 1 Comm: swapper/0
Call trace:
 dump_backtrace+0x0/0x1c0
 panic+0x120/0x2a0
 mount_block_root+0x1f4/0x2b0
 prepare_namespace+0x130/0x190
 kernel_init+0x10/0x100
```



Real Time

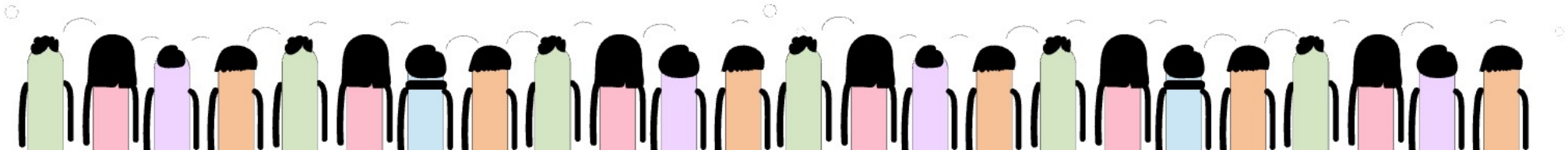
Log	Meaning
`Kernel panic`	Fatal kernel crash
`not syncing`	Cannot safely continue
`Unable to mount root fs`	Root filesystem missing
`PID 1`	init/systemd failed
`Call trace`	Stack trace



Real Time

REAL Production Causes

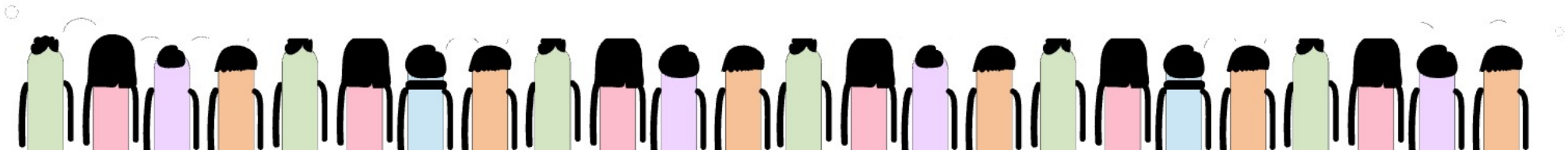
Cause	Example
Wrong rootfs	corrupted ext4
Missing driver	storage driver absent
Bad device tree	wrong hardware mapping
Kernel bug	null pointer
Memory corruption	invalid access



Real Time

Important Production Logs

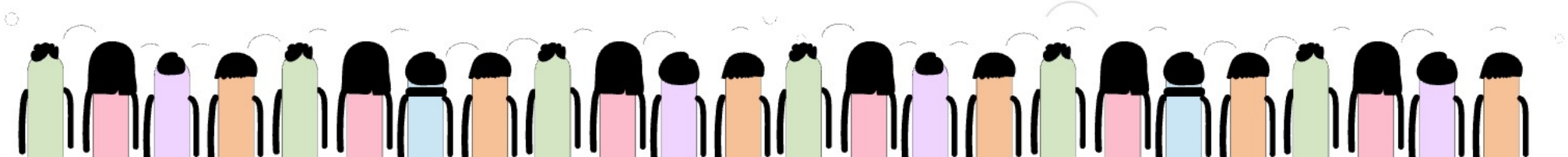
Command	Why Important
`dmesg`	kernel logs
`journalctl`	systemd logs
`systemctl status`	service state
`cat /proc/cmdline`	boot args
`lsmod`	loaded drivers
`mount`	filesystem status
`ip a`	network status



Day4 Flow

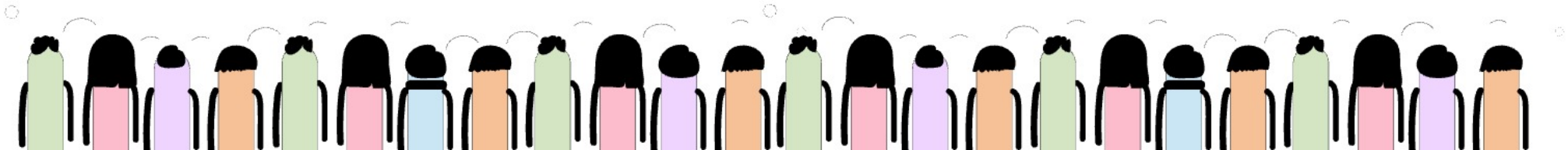
COMPLETE YOCTO MENTAL MODEL

```
Layers
   ↓
Recipes
   ↓
BitBake
   ↓
Cross Compilation
   ↓
Kernel Build
   ↓
BusyBox Build
   ↓
Root Filesystem
   ↓
Firmware Image
   ↓
QEMU Boot
```



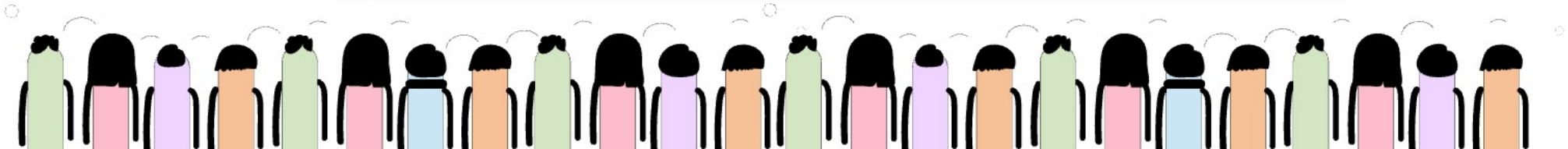
Day4 Flow

```
BitBake
    ↓
Reads recipes (.bb)
    ↓
Resolves dependencies
    ↓
Downloads source packages
    ↓
Cross compiles software
    ↓
Builds Linux kernel
    ↓
Builds BusyBox
    ↓
Creates root filesystem
    ↓
Generates firmware image
```



Day4 Flow

Concept	Meaning
Yocto	Embedded Linux build framework
Poky	Reference Yocto distro
BitBake	Build engine
Recipe	Package build instruction
Layer	Collection of recipes
RootFS	Embedded filesystem
QEMU	Hardware emulator
BusyBox	Embedded userspace
Cross Compilation	Build for another CPU
Firmware Image	Final e…bedded Linux OS



Q & A